NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 11: Understanding Program Efficiency II

Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Expose the work. Compare growth.
Measure fairly. Preserve the result.

SCHOOL OF ENGINEERING

About these notes

These notes accompany the 87-page Week 11 presentation, *Understanding Program Efficiency II*, developed earlier for ENG 101. They expand the same examples, vocabulary, numerical results, activities, and engineering cautions into a reading guide. The source deck, its earlier Python model module, and its recorded timing files determine the content; this is not a replacement topic sequence.

Course information

Course: ENG 101 — Programming for Engineers; lecture section 2L; 6 ECTS.
Instructor: Dr. Yanwei Wang, School of Engineering, Nazarbayev University.
Term: Fall 2026.
Week 11: Understanding Program Efficiency II — complexity classes and characteristic examples. The slides develop hidden work, recursive growth and memory, and a measured moving-window calculation. Their scope statement identifies CLOs 1 and 2.

Lecture-slide alignment: Week 11 source PDF, pp. 1–3.

Reading route

Source lecture	Central question
31: Hidden work in Python programs	Which scans, copies, shifts, and preparation steps are hidden behind a compact statement?
32: Recursive growth and memory	How much work occurs at every node, how many nodes are visited, and what remains alive simultaneously?
33: Scaling experiments and engineering choices	Does a proposed improvement preserve the required result, and what do counts and measurements actually establish?

Week 10 established input-size definitions, selected-operation counts, growth classes, and Big-O. Week 11 applies those ideas to complete implementations. Week 12 remains the dedicated searching-and-sorting implementation unit. The optional material in Chapter 4 stays optional; no new assessment or submission rule is introduced here.

Key idea

Expose the work → compare growth → measure fairly → preserve the result. A short program can hide a large computation. An improved algorithm must still satisfy its original contract, including data order, repeated records, physical units, and required accuracy.

How to use the examples

Before executing a program, identify the required result, the valid inputs, and each independently growing size. Trace a tiny example, then count the stated operation. Decide what data must be retained while the calculation is active. Only after that reasoning should you attach a complexity label. Make a prediction before reading each activity answer.

The numbered files in `examples/` are runnable demonstrations and worked completions. Keep `week11_models.py` beside them. Its 26 function/class definitions are the earlier slide companion implementations; they retain the same executable behavior. The notes print these definitions with comments, explain their statements in context, and add small driver programs to make their results observable. A code block marked as a model definition is not itself a complete console demonstration.

Source, explanation, and additions are distinguished

The source slides sometimes display only a function definition or import a helper. The recovered source module supplies those definitions. Added client calls, activity answers, instrumentation, tests, and cumulative exercises are worked additions for self-study, not claims that these lines appeared on the slides. Technical qualifications are explicitly marked where a brief slide statement needs a narrower interpretation.

The functions are *teaching implementations*. A stated precondition is not necessarily checked at runtime. For example, the moving-mean functions validate the window width, but assume that the supplied readings are finite numbers. The notes do not silently add a complete validation scan and then continue quoting the original operation counts.

Measurements are records, not universal constants

The original timing table and graph are retained in Chapter 3. Their raw trials, summary, and environment record are included unchanged. They describe one earlier run, not a new measurement on your computer. Compiling the notes never reruns a benchmark. The supplied optional benchmark tool writes a separately labelled new dataset instead of overwriting the source observations.

All other count tables and schematic plots are labelled mathematical or counting models. A ratio of selected operations is not a measured speedup. A hypothetical stage-time budget is not a benchmark result.

Two useful conventions

Source references use physical PDF pages 1–87, not the footer slide number. Divider pages make these numbering systems differ. Appendix A maps each physical source page to a reading section.

Memory categories are kept separate: supplied input, returned output, and auxiliary working storage. Numeric values and short identifiers use the bounded-cost classroom model unless otherwise stated. A count of scalar variables is not a claim that arbitrarily large Python integers occupy constant bytes.

Contents

About these notes 2

1. Hidden Work in Python Programs 6
1.1. Two loops do not settle the answer . . . 6
1.2. Write the analysis contract first . . . 7
1.3. Familiar operations and their hidden work . . . 7
1.4. Repeated concatenation copies growing prefixes . . . 8
1.5. Append reduces repeated work without changing the task . . . 9
1.6. Amortized growth: spread the occasional resize over the sequence . . . 10
1.7. Removing from the front repeats shifts . . . 11
1.8. One loop can contain a complete membership scan . . . 13
1.9. Construct and reuse a dictionary membership index . . . 14
1.10. Test what must remain unchanged . . . 15
1.11. Look inside methods and move only invariant work . . . 15
1.12. Sum dependent workloads instead of multiplying loop bounds . . . 17
1.13. Output requirements, recap, and exit-ticket answers . . . 19

2. Recursive Growth and Memory 20
2.1. Recursion is a program structure, not a complexity class . . . 20
2.2. Total nodes and maximum active frames are different counts . . . 20
2.3. Index-recursive summation: keep the same list . . . 21
2.4. Tail-slice summation: count what happens before the call . . . 22
2.5. Peak live memory: the waiting frames retain their tails . . . 22
2.6. Compare the sum implementations under one contract . . . 23
2.7. One halving call with constant local work . . . 24
2.8. A logarithmic number of calls can still copy linear data . . . 25
2.9. Follow one half, or process both disjoint halves . . . 26
2.10. Full local scans at every tree level . . . 27
2.11. Fibonacci: the subproblems overlap . . . 28
2.12. Memoization: make reused subproblems explicit . . . 30
2.13. The cache key must identify the same problem . . . 31
2.14. Worked activity, practical limits, and exit-ticket answers . . . 32

3. Scaling Experiments and Engineering Choices 33
3.1. Predict the scaling before measuring it . . . 33
3.2. Specify every full-window mean . . . 34
3.3. Direct calculation: recompute every window . . . 34
3.4. Reuse the overlap: derive the running update . . . 36
3.5. Test values, output length, boundaries, and non-mutation . . . 37
3.6. A faster algebraic update can lose required information . . . 38
3.7. Work, workspace, and returned output . . . 39
3.8. Plan the timing experiment around a fixed question . . . 39
3.9. Read the original measurements as a recorded experiment . . . 40
3.10. What the timing supports, and what it does not . . . 42
3.11. Operation budgets and multiple channels . . . 42
3.12. End-to-end speedup includes unchanged stages . . . 43
3.13. Review an improvement, prepare the lab, and report evidence . . . 43
3.14. Exit-ticket answers and the next unit . . . 44

4. Optional Extensions and Reference **45**
4.1. A bound for the toy resizing strategy . . . 45
4.2. Factorial growth: enumerating all orderings . . . 45
4.3. Preparation before repeated searches . . . 46
4.4. Reproducibility without silently changing the experiment . . . 47
4.5. Reference vocabulary and source reading . . . 47

5. Cumulative Practice and Worked Solutions **48**
5.1. Question 1: same number of loop headers . . . 48
5.2. Question 2: cumulative copies versus live storage . . . 48
5.3. Question 3: amortized is not average-case . . . 48
5.4. Question 4: zero comparisons do not mean zero work . . . 49
5.5. Question 5: repeated batches and stale indexes . . . 50
5.6. Question 6: removing an invariant calculation . . . 50
5.7. Question 7: distinguish two halving patterns . . . 50
5.8. Question 8: count recursive resources separately . . . 50
5.9. Question 9: fresh and warm Fibonacci computation . . . 51
5.10. Question 10: compare full-window calculations . . . 51
5.11. Question 11: explain the failed numerical update . . . 52
5.12. Question 12: distinguish models, observations, and conclusions . . . 52
5.13. Readiness checklist . . . 52

6. Synthesis and Glossary **53**
6.1. A complete explanation of efficiency . . . 53
6.2. Quick-reference comparisons . . . 53
6.3. Terminology . . . 54

A. Source Basis and Slide-by-Slide Coverage **56**
A.1. Source basis and technical references . . . 56
A.2. What was expanded or clarified . . . 56
A.3. Map every physical source page to a reading section . . . 57

Bibliography **59**

1 Hidden Work in Python Programs

Lecture-slide alignment: Week 11 source PDF, pp. 4–28.

Learning outcomes

By the end of this unit, you should be able to uncover scans, copies, and shifts; retain independent input dimensions; distinguish amortized costs from average-case claims; include preprocessing and output storage; and verify that an improvement preserves the original result.

1.1. Two loops do not settle the answer

Lecture-slide alignment: Week 11 source PDF, pp. 4–6.

The number of loop headers is not a complexity formula. The amount of work depends on how often their bodies execute and what the bodies do. The source warm-up compares a fixed inner bound with a bound that grows with the outer index.

```
# ENG 101 | Week 11 | Fixed and triangular inner work
# Worked activity solution: p. 6.
# Initialize independent counters before their respective loops.
n = 4
fixed = 0
triangular = 0
for i in range(n):
    for j in range(3):
        fixed += 1  # Three visits for each outer pass.
for i in range(n):
    for j in range(i):
        triangular += 1  # The inner bound depends on the outer index.
print(fixed, triangular)
```

Code 1.1: Worked warm-up: initialize each counter and count body visits

```
12 6
```

For the first loop, every one of the four outer passes causes three visits, so the count is $4 \times 3 = 12$. In general, it is $3n$, giving linear growth. In the second loop, the inner range has lengths $0, 1, 2, 3$, giving six visits. For arbitrary nonnegative integer n,

$$0 + 1 + \cdots + (n-1) = \frac{n(n-1)}{2}.$$

The second count therefore grows quadratically. The counter counts selected body visits, not every assignment, loop-condition check, or machine instruction executed by Python.

1.2. Write the analysis contract first

Lecture-slide alignment: Week 11 source PDF, p. 7.

A useful efficiency claim is a sentence about a particular task, not just a label such as "linear." Before analyzing an implementation, specify what may grow, what must be returned, and what operations are being counted.

Part	Approval-check example
Input dimensions	q installed identifiers and m approved identifiers.
Selected operation	Equality comparisons between short identifiers.
Input case	For the worst-case example, every installed identifier is absent.
Required result	One Boolean flag per installed record, preserving order and repetition.
Cost assumptions	Short-string equality and ordinary hashing have bounded cost.
Included preparation	State whether the approved lookup table has already been constructed.

The numbers q and m are independent. A large installation can have a short approved list; a small installation can consult a large one. Replacing both by n too early hides this distinction. Likewise, a function returning *all flags* cannot stop at the first failed lookup: that would answer a different question.

Engineering check

Keep the meaning of the data visible. An identifier such as `T-2` is text representing an entity, not a numerical quantity. Repeated installed identifiers can represent separate records that must each receive a result.

1.3. Familiar operations and their hidden work

Lecture-slide alignment: Week 11 source PDF, p. 8; p. 79.

For an existing Python list of length n, the slides use the following implementation-aware model. Costs refer to ordinary bounded-cost element operations, not arbitrary objects with expensive custom methods.

Operation	Growth model	Work to explain
`len(a)`, `a[i]`	$O(1)$	Read a stored length or a known position.
`sum(a)`	$O(n)$	Accumulate every supplied number.
`x in a`	$O(n)$ worst case	Search until a match is found or all positions are exhausted.
`a[:k]`	$O(k)$	Copy the selected references into a new list.
`a.copy()`	$O(n)$	Copy all references into a new outer list.
`a.insert(0, x)`	$O(n)$	Shift the existing references to make space.
`a.append(x)`	$O(1)$ amortized	Add at the end; some calls need capacity growth.

Interpret the slice cost using the number actually copied

The table's k is the number of selected elements, or a positive in-range prefix length. A slice whose stop is beyond the list copies only the available elements. Negative bounds also need interpretation before counting. More generally, copying a slice of length r costs work proportional to r, with constant setup even when $r = 0$.

Copying a list is *shallow*: the new outer list contains references to the same element objects. It does not recursively duplicate every nested object. Also, the syntax of an operation does not guarantee its cost. A method on a user-defined class can perform a complete computation before returning a single value.

1.4. Repeated concatenation copies growing prefixes

Lecture-slide alignment: Week 11 source PDF, pp. 9–10.

The following source function doubles each value, keeps the original order, and leaves its input unchanged. It produces the correct small result, but repeatedly constructs a replacement output list.

```
# Source teaching implementation; see the surrounding contract.
def build_by_concatenation(values):
    """Double each value; preserve order and the input list."""
    result = []
    for value in values:
        # This creates a new list and recopies the growing prefix.
        result = result + [2 * value]
    return result
```

Code 1.2: Source function: a new list is built on every pass

The expression `[2 * value]` first forms a one-element list. The `+` operator then combines the existing result with that new list. Unlike numerical addition, list concatenation must populate a new list with references from both operands. Assignment rebinds `result` to the new object; it does not extend the previous object in place.

```python
# ENG 101 | Week 11 | Repeated prefix copying
# Source demonstration: pp. 9-10.
from week11_models import build_by_concatenation
# A new result is requested; the original values must remain intact.
values = [2, 3, 4]
print(build_by_concatenation(values))
print(values)
```

Code 1.3: Small client demonstration of the source function

```
[4, 6, 8]
[2, 3, 4]
```

Pass	Old length	New result length	Cumulative result slots populated
1	0	1	1
2	1	2	3
3	2	3	6
4	3	4	10

The selected count is

$$C(n) = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} = \Theta(n^2).$$

This count includes slots populated in the new concatenated results. It does not separately count temporary singleton creation, multiplication, loop control, or assignments. Those additional operations do not remove the quadratic copying term.

Cumulative allocation is not peak live storage

Earlier result lists are not all retained by this program. During a replacement, the old and new lists can coexist temporarily, but obsolete earlier results are not saved. The peak list storage grows linearly, even though the total copying over the run is quadratic.

1.5. Append reduces repeated work without changing the task

Lecture-slide alignment: Week 11 source PDF, p. 11.

```python
# Source teaching implementation; see the surrounding contract.
def build_by_append(values):
    """Return the same doubled values without repeated prefix copies."""
    result = []
    for value in values:
        # Append one new value to the existing output list.
        result.append(2 * value)
    return result
```

Code 1.4: Source function: grow one output list

`result.append(2 * value)` places one newly computed value at the end of the existing output. It changes that output list and returns `None`; the function therefore returns `result` after the loop, rather than assigning the return of `append()`.

```python
# ENG 101 | Week 11 | Append instead of rebuilding the prefix
# Source demonstration: p. 11.
from week11_models import build_by_append
# Appending changes only the newly created result list.
values = [2, 3, 4]
print(build_by_append(values))
print(values)
```

Code 1.5: The improved function retains order, values, and input preservation

```
[4, 6, 8]
[2, 3, 4]
```

There are n append calls, one per input value. Their total list-growth work is linear in the usual amortized model. The output still contains n values and still requires linear storage. Removing repeated copying does not eliminate the information that the caller requested.

1.6. Amortized growth: spread the occasional resize over the sequence

Lecture-slide alignment: Week 11 source PDF, pp. 12–13.

The source illustrates append behavior with a *toy doubling-capacity strategy*. A storage array starts with zero capacity. When full, it grows to one slot or doubles its capacity, copying the existing references. Every append then writes one new value.

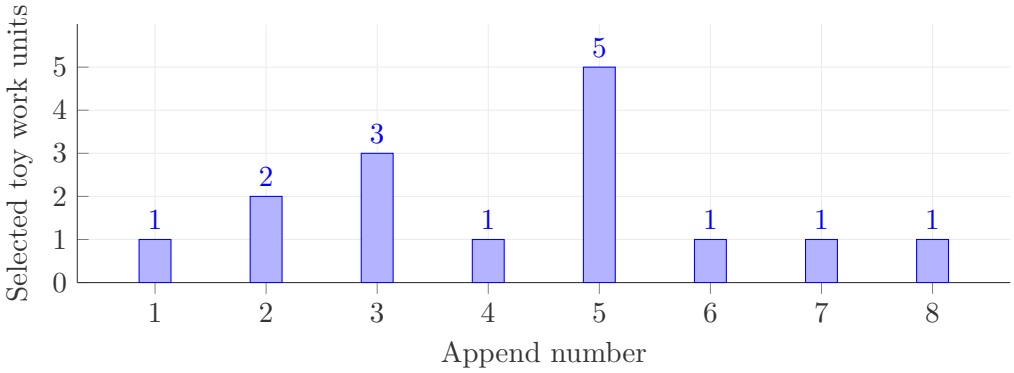


Figure 1.1.: The source's toy doubling-capacity model: a new-value write plus any copies of existing entries. The bars are model counts, not CPython allocation measurements or timings. Source PDF p. 12; redrawn.

For the first eight appends, the selected costs are $1, 2, 3, 1, 5, 1, 1, 1$. The expensive third and fifth appends include copies of two and four old references. The total is

$$8 + (1 + 2 + 4) = 15$$

selected operations: eight new-value writes and seven copied old references.

```python
# Source teaching implementation; see the surrounding contract.
def toy_capacity_cost(n):
    """Model n appends with doubling capacity; NOT CPython's growth rule."""
    capacity = 0
    copied = 0
    for size in range(n):
        if size == capacity:
            # A conceptual resize copies all existing references.
            copied += size
            capacity = max(1, 2 * capacity)
    return copied, n  # Resize copies and new-value writes are distinct.
```

Code 1.6: Source counting model: track capacity changes, not an actual Python allocation

```python
# ENG 101 | Week 11 | Toy doubling-capacity cost
# Worked model illustration: pp. 12, 80.
from week11_models import toy_capacity_cost
# Count copied old entries separately from the new-value writes.
for n in [1, 2, 3, 4, 8]:
    copies, writes = toy_capacity_cost(n)
    print(n, copies, writes, copies + writes)
```

Code 1.7: Worked use of the toy counting model

```
1 0 1 1
2 1 2 3
3 3 3 6
4 3 4 7
8 7 8 15
```

The four printed columns are number of appends, old references copied, new-value writes, and their total. This function simulates a capacity policy with integer counters; it does not inspect Python's allocator or measure time.

Average-case versus amortized

An **average-case** claim averages over a specified input distribution or relevant randomized behavior. An **amortized** claim bounds work over a whole sequence of operations, including occasional expensive ones. The toy capacity argument requires no random input distribution.

The distinction matters in reports. The slides qualify append as amortized $O(1)$ and dictionary lookup as average/expected $O(1)$ under suitable hashing assumptions. Neither statement says that every individual operation has the same cost, and neither is a universal worst-case guarantee for arbitrary keys or implementations. The doubling schedule is an illustration, *not CPython's exact allocation rule.*

1.7. Removing from the front repeats shifts

Lecture-slide alignment: Week 11 source PDF, p. 14.

```python
# Source teaching implementation; see the surrounding contract.
def drain_front(values):
    """Sum a COPY by removing its first item; use small integers here."""
    work = values.copy()  # Preserve the caller's list.
    total = 0
    while work:
        # Removing index 0 shifts the remaining references left.
        total += work.pop(0)
    return total
```

Code 1.8: Source function: remove from the beginning of a copied list

```python
# Source teaching implementation; see the surrounding contract.
def drain_back(values):
    """Sum a COPY by removing its last item; use small integers here."""
    work = values.copy()
    total = 0
    while work:
        # No later elements must be shifted after removing the last item.
        total += work.pop()
    return total
```

Code 1.9: Source function: remove from the end of a copied list

Both functions start with a copy to preserve the caller's list. `while work` continues while that copy is nonempty. `pop(0)` returns and removes the first value, requiring subsequent references to shift left. `pop()` returns and removes the last value, with no following elements to shift.

```python
# ENG 101 | Week 11 | Front and back removal
# Source demonstration: p. 14.
from week11_models import drain_front, drain_back
# Small integers avoid floating-point ordering effects in this comparison.
values = [2, 3, 4]
print(drain_front(values))
print(drain_back(values))
print(values)  # Both functions operate on copies.
```

Code 1.10: Compare both functions on exactly summable small integers

```
9
9
[2, 3, 4]
```

For front removal, the shift counts are $n-1, n-2, \ldots, 0$, totaling $n(n-1)/2$. Thus the complete front-draining algorithm is quadratic. End removal avoids those repeated shifts, giving linear amortized total work, including the initial copy and all pops. Both require a copy of length n at the start.

Engineering check

Front and back removal sum values in different orders. The small integer demonstration has the same exact sum. Floating-point inputs can produce different rounding errors; agreement for the example is not a guarantee for every numerical dataset. Neither algorithm is a recommendation to delete a record that must remain available.

1.8. One loop can contain a complete membership scan

Lecture-slide alignment: Week 11 source PDF, pp. 15–16.

```python
# Source teaching implementation; see the surrounding contract.
def approval_flags_list(installed, approved):
    """One Boolean per installed ID, retaining its order and duplicates."""
    flags = []
    for sensor_id in installed:
        # With a list, this membership test may scan all approved IDs.
        flags.append(sensor_id in approved)
    return flags
```

Code 1.11: Source function: one Boolean per installed record

`sensor_id in approved` performs a list membership search. In the all-absent case, each of the q installed records requires m equality comparisons, for qm comparisons overall. If every match is at the first approved position and $m \geq 1$, only q comparisons are needed. The same code and input lengths can therefore take different paths.

```python
# ENG 101 | Week 11 | One flag per installed record
# Source demonstration: pp. 15-16.
from week11_models import approval_flags_list
# Repeated installed identifiers represent separate records.
installed = ["T-2", "T-9", "T-2"]
approved = ["T-2", "T-2", "T-1"]
print(approval_flags_list(installed, approved))
print(approval_flags_list(installed, []))
print(approval_flags_list([], approved))
```

Code 1.12: Normal, empty-approved, and empty-installed examples

```
[True, False, True]
[False, False, False]
[]
```

The first output preserves the repeated `T-2` record. The second has three false flags even though no ID equality comparisons are possible: producing the flags is still work.

For $q \geq 1$, a useful complete worst-case expression is

$$T(q, m) = \Theta(q(m + 1)).$$

The extra 1 captures output and loop work when $m = 0$. When $q = 0$, there are no membership tests and the function has constant setup and return work. If $q = m = n$ grows, the worst case is quadratic. It is incorrect to infer that all two-list problems are quadratic without examining their separate sizes and behavior.

1.9. Construct and reuse a dictionary membership index

Lecture-slide alignment: Week 11 source PDF, pp. 17–19.

```python
# Source teaching implementation; see the surrounding contract.
def build_approval_lookup(approved):
    """Build a membership index for short, hashable string identifiers."""
    lookup = {}
    for sensor_id in approved:
        # Repeated approved IDs need only one entry for membership.
        lookup[sensor_id] = True
    return lookup
```

Code 1.13: Source function: construct a dictionary of approved identifiers

Each approved identifier becomes a dictionary key. The value `True` is only a marker; membership queries inspect keys, not values. Repeated approved keys overwrite the same marker, which is harmless when the question is simply whether an identifier is approved.

If $u \leq m$ is the number of distinct approved identifiers, construction retains u entries. With short, hashable IDs and ordinary hash behavior, expected construction work is $O(m)$.

```python
# Source teaching implementation; see the surrounding contract.
def approval_flags_lookup(installed, lookup):
    """Return flags using an ALREADY BUILT dictionary index."""
    flags = []
    for sensor_id in installed:
        # Membership checks dictionary keys, not dictionary values.
        flags.append(sensor_id in lookup)
    return flags
```

Code 1.14: Source function: query an already constructed index

```python
# ENG 101 | Week 11 | Build once and retain the output contract
# Source demonstration: pp. 17-19.
from week11_models import build_approval_lookup, approval_flags_lookup
# Duplicate approved keys collapse, but installed records do not.
installed = ["T-2", "T-9", "T-2"]
approved = ["T-2", "T-2", "T-1"]
lookup = build_approval_lookup(approved)
print(lookup)
print(approval_flags_lookup(installed, lookup))
```

Code 1.15: Construct once, then produce one flag per installed record

```
{'T-2': True, 'T-1': True}
[True, False, True]
```

Expected work for a complete first batch is $O(m+q)$: build the index, then perform q membership queries and output writes. Extra storage includes the $O(u)$ index, while the required output has $O(q)$ flags.

Plan for B batches, each with q installed records	Cost model
Scan the approved list on every query	$O(Bq(m+1))$ worst case
Rebuild the dictionary for each batch	$O(B(m+q))$ expected
Build once and reuse it for all batches	$O(m+Bq)$ expected

This comparison assumes the approved data remain applicable. When they change, the lookup must be updated or rebuilt. A stale index can make a fast but incorrect decision. If every batch's flags are retained, output storage grows with Bq; processing one batch at a time has a different retained-output requirement.

1.10. Test what must remain unchanged

Lecture-slide alignment: Week 11 source PDF, p. 20.

```python
# ENG 101 | Week 11 | Regression checks after changing representation
# Worked completion of source tests: p. 20.
from week11_models import build_approval_lookup, approval_flags_lookup
# Independently known expected flags preserve order and repetition.
installed = ["T-2", "T-9", "T-2"]
approved = ["T-2", "T-2", "T-1"]
original = installed.copy()
lookup = build_approval_lookup(approved)
assert approval_flags_lookup(installed, lookup) == [True, False, True]
assert installed == original
assert approval_flags_lookup([], lookup) == []  # No installed records.
assert approval_flags_lookup(installed, {}) == [False, False, False]
print("Approval checks passed")
```

Code 1.16: Source tests with explicitly added empty-case checks

```
Approval checks passed
```

The known output `[True, False, True]` checks values, order, and repetition. The saved copy checks non-mutation. Additional useful cases are a singleton match, all matches, no matches, repeated approved identifiers, and a changed approved collection. These cases examine the contract independently of the data structure used to implement it.

Common mistake

Replacing the result with one Boolean for each *distinct* installed identifier would appear to reduce work but would remove required records. An optimization must not quietly discard duplicates, metadata, or units.

1.11. Look inside methods and move only invariant work

Lecture-slide alignment: Week 11 source PDF, pp. 21–23.

```python
# Source teaching implementation; see the surrounding contract.
class ReadingLog:
    """An existing nonempty numeric record; units belong to its contract."""

    def __init__(self, readings):
        if not readings:
            raise ValueError("at least one reading is required")
        self.readings = readings.copy()  # Construction costs O(n).

    def latest(self):
        # One positional lookup on an existing list.
        return self.readings[-1]

    def mean(self):
        # A short method call can hide a complete scan.
        return sum(self.readings) / len(self.readings)
```

Code 1.17: Source class: construction, one-position lookup, and a full-scan mean

The initializer rejects an empty record, then copies the input. After construction, `latest()` reads one stored position. `mean()` calls `sum()` and therefore visits all readings. The same short dot-call syntax can invoke very different amounts of work.

```python
# ENG 101 | Week 11 | Methods with different hidden costs
# Source demonstration: p. 21.
from week11_models import ReadingLog
# Creating this log copies the input; each query has its own cost.
values = [20.0, 22.0, 24.0]
log = ReadingLog(values)
print(log.latest())
print(log.mean())
print(log.readings == values, log.readings is values)
```

Code 1.18: Inspect results and the distinction between equal and identical lists

```
24.0
22.0
True False
```

The final output says the two lists contain equal values but are different outer objects. This is why the class can have a separate stored list after initialization.

```python
# Source teaching implementation; see the surrounding contract.
def centre_repeated(readings):
    """Subtract the same mean repeatedly; intentionally inefficient."""
    centred = []
    for value in readings:
        # The unchanged input is scanned again on every pass.
        mean = sum(readings) / len(readings)
        centred.append(value - mean)
    return centred
```

Code 1.19: Source function: intentionally recompute an unchanged mean

For every output element, `sum(readings)` revisits all n readings. There are n such scans, giving quadratic work. The input is not modified inside the function, so its mean remains unchanged throughout.

```python
# Source teaching implementation; see the surrounding contract.
def centre_once(readings):
    """Move an invariant calculation outside the output loop."""
    if not readings:
        return []
    mean = sum(readings) / len(readings)
    centred = []
    for value in readings:
        # The mean is valid while the readings remain unchanged.
        centred.append(value - mean)
    return centred
```

Code 1.20: Source function: compute the invariant mean once

The empty case must now be handled before division: the old version skipped its body when empty, whereas an unguarded hoisted mean would divide by zero. For nonempty inputs, the new version adds one mean-computation scan and one output scan, giving linear total work.

```python
# ENG 101 | Week 11 | Move unchanged work outside the loop
# Source demonstration: pp. 22-23.
from week11_models import centre_repeated, centre_once
# Both implementations subtract the same mean from the unchanged data.
values = [20.0, 22.0, 24.0]
print(centre_repeated(values))
print(centre_once(values))
print(centre_once([]))  # The empty-input case remains defined.
```

Code 1.21: The centering results remain the same

```
[-2.0, 0.0, 2.0]
[-2.0, 0.0, 2.0]
[]
```

Hoisting a calculation is valid only if its inputs remain unchanged and it has no essential side effects that must occur on every pass. An operation that reads a changing sensor, updates a state, or uses changing global data cannot be moved merely because its source text repeats.

1.12. Sum dependent workloads instead of multiplying loop bounds

Lecture-slide alignment: Week 11 source PDF, pp. 24–25.

```python
# Source teaching implementation; see the surrounding contract.
def geometric_work(n):
    """Count visits for full, half, quarter, ... workloads; n >= 0."""
    visits = 0
    size = n
    while size > 0:
        for _ in range(size):
            visits += 1  # One selected visit, not one machine instruction.
        size //= 2
    return visits
```

Code 1.22: Source counting pattern: the inner workload shrinks at every stage

For $n = 8$, the loop visits $8 + 4 + 2 + 1 = 15$ positions. For any positive integer n,

$$n \leq n + \lfloor n/2 \rfloor + \lfloor n/4 \rfloor + \cdots < 2n.$$

Both a linear lower bound and a linear upper bound hold, so total work is $\Theta(n)$. At powers of two the exact count is $2n - 1$.

```python
# Source teaching implementation; see the surrounding contract.
def full_scan_each_stage(n):
    """Count n visits at each halving stage; n >= 1."""
    visits = 0
    size = n
    while size > 1:
        for _ in range(n):
            visits += 1  # Work stays n even as the stage counter shrinks.
        size //= 2
    return visits
```

Code 1.23: Source activity: the stage counter shrinks but each scan remains full size

Here the inner range is always `range(n)`, not `range(size)`. At $n = 8$, the size values entering the loop are $8, 4, 2$: three stages, each with eight visits. The result is 24 rather than 15.

```python
# ENG 101 | Week 11 | Shrinking local work versus full repeated scans
# Worked activity solution: pp. 24-25.
from week11_models import geometric_work, full_scan_each_stage
# The two functions have different inner-loop bounds.
n = 8
print(geometric_work(n))
print(full_scan_each_stage(n))
```

Code 1.24: Worked answer to the contrasting-stage activity

```
15
24
```

For $n \geq 1$, the second counter is $n\lfloor \log_2 n \rfloor$. For growing n its work is $\Theta(n \log n)$. A similar halving outer loop can therefore give linear or log-linear work, depending on what happens inside it. For $n = 1$, its selected counter is zero, but the function still performs constant setup and return operations.

1.13. Output requirements, recap, and exit-ticket answers

Lecture-slide alignment: Week 11 source PDF, pp. 26–28.

Returning n corrected readings requires producing n separate results. Returning a single warning count can use scalar output, but it answers a different question. Compare implementations that preserve values, order, multiplicity, units, and documented failure behavior.

Section summary

An effective hidden-work review states the growing dimensions, expands scans and allocations, sums dependent workloads, qualifies hashing and amortization, separates preprocessing and output storage, and repeats the correctness tests after the change.

Exit question 1: Why can a list-membership expression inside one loop be quadratic? The outer loop may make q queries, each scanning m approved elements. When both sizes grow like n, the all-absent comparison count is n^2. The membership expression is not constant-time merely because it is one expression.

Exit question 2: What does amortized constant append cost not promise? It does not promise an identical cost for every append, a fixed elapsed duration, or a worst-case constant-time individual resize. It bounds the work across a sequence under the stated list-growth model.

Exit question 3: For q flags and m approved IDs, how does list lookup compare with a dictionary built once? List lookup has worst-case $\Theta(q(m+1))$ work for $q \geq 1$. The dictionary plan includes expected $O(m)$ construction for the first use, expected $O(q)$ query/output work, retained distinct-key storage, and the required flags. State whether the index is already available, whether it remains current, and whether previous output batches remain stored.

2 Recursive Growth and Memory

Lecture-slide alignment: Week 11 source PDF, pp. 29–53.

Learning outcomes

By the end of this unit, you should be able to distinguish total calls from maximum active depth; count local copying; explain one-half, disjoint-half, and overlapping subproblems; compare recursion with iteration and caching; and state separate time and storage conclusions.

2.1. Recursion is a program structure, not a complexity class

Lecture-slide alignment: Week 11 source PDF, pp. 29–31.

The word *recursive* does not determine whether a program is logarithmic, linear, quadratic, or exponential. Four questions are needed: how many child calls are made, how large their inputs are, what local work occurs before and after those calls, and what remains stored while calls wait.

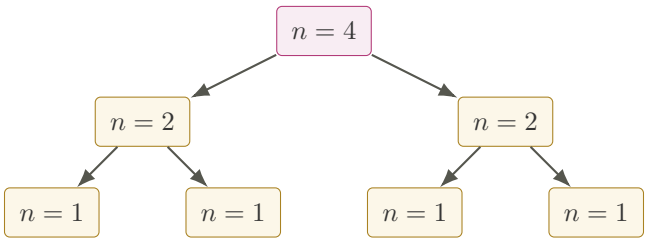
A single call on $n - 1$ with constant local work usually produces a linear chain. A single call on $\lfloor n/2 \rfloor$ with constant local work gives logarithmically many levels. Two calls on disjoint halves can still have only linear total work. A new tail slice on every level can change an otherwise linear chain into a quadratic computation.

Local work

Local work is the work performed in one invocation, excluding the work done inside its child invocations. It includes preparing the child arguments. Creating a list slice before a recursive call is therefore local work; it does not disappear into the notation for a smaller problem.

2.2. Total nodes and maximum active frames are different counts

Lecture-slide alignment: Week 11 source PDF, p. 32.



Seven calls in total; only three frames on a deepest active path.

Figure 2.1.: A complete two-half call tree for four leaves. Total visited nodes and maximum active frames count different resources. Source PDF p. 32; redrawn.

The illustrated tree processes four leaves. There are seven calls in the entire execution: the root, two intermediate calls, and four leaves. During ordinary depth-first execution, however, only one branch is active at a time. The deepest active chain contains three frames.

A frame retains the parameters, local names, and continuation information for one unfinished invocation. Once the first child returns, its frame can finish before the second child is called. Counting every node in the tree as simultaneous storage would overestimate stack use.

State the depth convention

A longest root-to-leaf path in this picture has two *edges*, but three *active call frames*. The notes count frames when discussing stack storage. The two conventions differ by one, not by their asymptotic growth.

Time depends on local work summed over all visited nodes. Peak stack storage depends on the largest active chain. Additional lists or caches may require more memory than the frames themselves, so stack depth is not always the full storage answer.

2.3. Index-recursive summation: keep the same list

Lecture-slide alignment: Week 11 source PDF, p. 33.

```
# Source teaching implementation; see the surrounding contract.
def sum_by_index(values, index=0):
    """Sum a short list recursively without copying tails; start at 0."""
    if index == len(values):
        return 0
    # Pass the same list and a new position to the smaller subproblem.
    return values[index] + sum_by_index(values, index + 1)
```

Code 2.1: Source function: pass an index instead of copying a tail

The entry call uses `index=0`. Each non-base invocation reads `values[index]` and calls the function with the same list and the next index. The base condition `index == len(values)` represents an empty remaining interval and returns zero.

For $[1, 2, 3, 4]$, the unfinished expression expands as

$$1 + (2 + (3 + (4 + 0))) = 10.$$

There are n non-base calls, n selected additions, and one base call: $n + 1$ calls in total. A recurrence describing the work is

$$T(n) = T(n - 1) + \Theta(1), \qquad T(0) = \Theta(1).$$

Thus total work is $\Theta(n)$ for growing n. The maximum active chain also has $n + 1$ frames, so auxiliary stack storage is $\Theta(n)$ in the classroom model.

Function contract

Use a short numeric list and enter at index zero. The function does not validate an arbitrary caller-supplied index. A negative or excessively large index is outside the demonstrated entry contract. No hidden validation pass is included in the quoted counts.

2.4. Tail-slice summation: count what happens before the call

Lecture-slide alignment: Week 11 source PDF, pp. 34–35.

```python
# Source teaching implementation; see the surrounding contract.
def sum_by_slicing(values):
    """Sum a short list recursively; each tail slice allocates a list."""
    if not values:
        return 0
    # Copies len(values)-1 references before the next call.
    return values[0] + sum_by_slicing(values[1:])
```

Code 2.2: Source function: every smaller problem receives a copied tail

The mathematical decomposition resembles the index version. However, evaluating `values[1:]` creates a new outer list before the next invocation starts. For an input of length n, that list has $n - 1$ references.

Current length	Next slice length	Copies so far	Next action
4	3	3	Recurse on the three-element tail.
3	2	5	Recurse on the two-element tail.
2	1	6	Recurse on the singleton.
1	0	6	Recurse on the empty list.
0	—	6	Return the base value.

The source's selected copy count is

$$C_{\text{copy}}(n) = \sum_{j=0}^{n-1} j = \frac{n(n-1)}{2}.$$

The function still makes $n + 1$ calls and performs n additions. Counting only those additions misses the dominant work. With copying included,

$$T(n) = T(n-1) + \Theta(n) = \Theta(n^2).$$

The copied references are not deep copies of the numeric objects. Nevertheless, creating all those reference slots is real work.

2.5. Peak live memory: the waiting frames retain their tails

Lecture-slide alignment: Week 11 source PDF, p. 36.

In this implementation, every unfinished caller retains its own `values` argument. At the deepest descent, the copied lists of lengths $n-1, n-2, \ldots, 1, 0$ can therefore coexist. Excluding the original input, their slots total $n(n-1)/2$, and their list headers and the active frames add linear overhead.

The peak extra referenced-list storage is consequently $\Theta(n^2)$ for this particular implementation. This conclusion is stronger than merely saying that the recursion depth is linear.

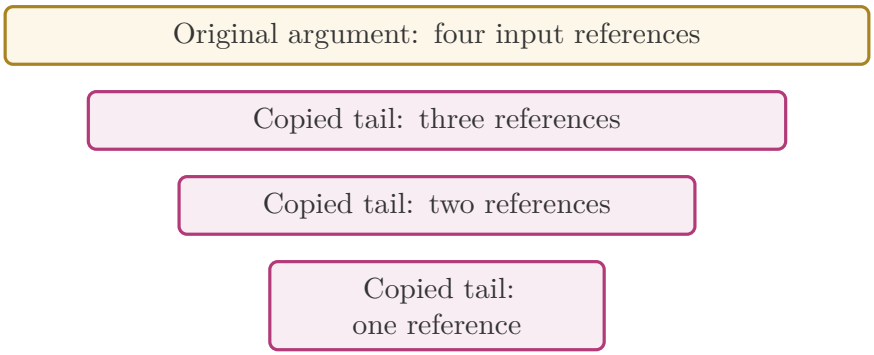


Figure 2.2.: Live arguments during the tail-slice recursion. The original input is shown for context but excluded from the extra-copy count. Empty-list and frame overhead are additional. Source PDF p. 36; redrawn.

Why this differs from repeated iterative concatenation

The iterative concatenation example does not retain all earlier result lists. The recursive tail-slice example retains earlier tails through unfinished call frames. Both perform quadratic cumulative copying, but their *peak live* list storage differs because the references that remain reachable differ.

2.6. Compare the sum implementations under one contract

Lecture-slide alignment: Week 11 source PDF, p. 37.

```python
# Source teaching implementation; see the surrounding contract.
def sum_iteratively(values):
    """Sum once using a running scalar, without copying the input."""
    total = 0
    for value in values:
        total += value  # One arithmetic update per element.
    return total
```

Code 2.3: Source function: one running subtotal

```python
# ENG 101 | Week 11 | Three implementations of a small integer sum
# Source demonstration: pp. 33-37.
from week11_models import sum_by_index, sum_by_slicing, sum_iteratively
# Use a short list: a linear recursive chain is not for large logs.
values = [1, 2, 3, 4]
for method in [sum_by_index, sum_by_slicing, sum_iteratively]:
    print(method(values), method([]))
print(values)  # Every implementation preserves the input.
```

Code 2.4: Small integer examples for the three source functions

```
10 0
10 0
10 0
[1, 2, 3, 4]
```

The six numerical outputs are pairs of a normal-case result and an empty-case result for each method. The last line confirms the unchanged input. All three functions return one scalar sum, but their resource use differs.

Method	Time model	Auxiliary storage
Index recursion	$\Theta(n)$	$\Theta(n)$ frames.
Tail-slice recursion	$\Theta(n^2)$	$\Theta(n^2)$ retained list references plus frames.
Running subtotal	$\Theta(n)$	A constant number of scalar variables.

These claims use bounded-cost values and operations. The recursive versions group additions differently from a left-to-right loop. Exact small-integer agreement does not imply identical floating-point results for all inputs. The source deliberately uses small values so that the resource comparison does not become confused with a change in numerical behavior.

2.7. One halving call with constant local work

Lecture-slide alignment: Week 11 source PDF, p. 38.

```python
# Source teaching implementation; see the surrounding contract.
def halving_depth(n):
    """Return the number of floor-halvings to reach 1; integer n >= 1."""
    if n <= 1:
        return 0
    # One recursive call on approximately half the previous size.
    return 1 + halving_depth(n // 2)
```

Code 2.5: Source function: return the number of floor-halving reductions

For $n = 16$, the values are $16, 8, 4, 2, 1$. Four reductions occur, but five calls occur because the final base invocation also executes. For positive integer n, the returned reduction count is $\lfloor \log_2 n \rfloor$.

```python
# ENG 101 | Week 11 | Count reductions, not the base call
# Source demonstration: p. 38.
from week11_models import halving_depth
# Each result counts integer floor-halvings needed to reach one.
for n in [1, 3, 8, 16]:
    print(n, halving_depth(n))
```

Code 2.6: Worked calls distinguish reductions from total invocations

```
1 0
3 1
8 3
16 4
```

The model recurrence is

$$T(n) = T(\lfloor n/2 \rfloor) + \Theta(1).$$

For large n, both time and the maximum number of frames grow as $\Theta(\log n)$. At $n = 1$, the return value is zero but the function still costs one base call. The function's precondition is $n \geq 1$; its use of `n <= 1` does not make every smaller or noninteger argument a meaningful problem instance.

2.8. A logarithmic number of calls can still copy linear data

Lecture-slide alignment: Week 11 source PDF, p. 39.

```python
# Source teaching implementation; see the surrounding contract.
def first_by_half_slicing(values):
    """A deliberately wasteful first-item retrieval for cost analysis."""
    if not values:
        raise ValueError("a nonempty list is required")
    if len(values) == 1:
        return values[0]
    # Few calls do not eliminate the cost of copying each half-list.
    return first_by_half_slicing(values[:len(values) // 2])
```

Code 2.7: Source analysis example: wasteful retrieval through copied halves

This function repeatedly keeps the first half, so it eventually returns the first element. It is intentionally inefficient for that task: direct access `values[0]` already provides the same value.

```python
# ENG 101 | Week 11 | Logarithmic call count can hide linear copying
# Source demonstration: p. 39.
from week11_models import first_by_half_slicing
# Deliberately inefficient analysis example, not recommended retrieval.
values = [10, 20, 30, 40, 50, 60, 70, 80]
print(first_by_half_slicing(values))
print(values[0])  # The direct solution has the same returned value.
```

Code 2.8: Two implementations with the same tiny retrieval result

```
10
10
```

For a power-of-two length n, only $\log_2 n + 1$ calls are made. Yet the copied half-list lengths sum to

$$\frac{n}{2} + \frac{n}{4} + \cdots + 1 = n - 1.$$

Total time is linear, not logarithmic. Because waiting frames retain the copied halves, extra list storage is also linear; the stack alone is logarithmic. For general positive sizes, floor division changes the exact sum but not the linear growth.

Common mistake

"The input is halved, therefore the whole function is logarithmic" ignores the cost of constructing each halved input. A constant-cost change of index bounds and a linear-cost list slice are different operations.

2.9. Follow one half, or process both disjoint halves

Lecture-slide alignment: Week 11 source PDF, pp. 40–41.

With constant local work and no copying, following one branch gives

$$T(n) = T(n/2) + c,$$

whereas processing both halves gives

$$T(n) = 2T(n/2) + c.$$

The first visits one node per level. The second visits a complete tree. For n leaves, a full binary tree has $n - 1$ internal nodes and $2n - 1$ total nodes. It can have logarithmic depth and linear total work at the same time.

```python
# Source teaching implementation; see the surrounding contract.
def balanced_sum(values, left=0, right=None):
    """Sum by dividing an index interval; no list slices are created."""
    if right is None:
        right = len(values)
    if left == right:
        return 0  # Empty interval.
    if right - left == 1:
        return values[left]  # One leaf value.
    middle = (left + right) // 2
    # Two disjoint subproblems, then one addition at this node.
    return (balanced_sum(values, left, middle)
            + balanced_sum(values, middle, right))
```

Code 2.9: Source function: balanced summation with index bounds

`left` is included and `right` is excluded. At the initial call, `right=None` is replaced by `len(values)`. Child calls then receive explicit integer endpoints. An empty interval returns zero; an interval of length one returns its leaf value. Each internal call adds the sums from two nonempty disjoint intervals.

```python
# ENG 101 | Week 11 | Two disjoint index intervals
# Source demonstration: pp. 40-41.
from week11_models import balanced_sum
# Both halves are processed, with no new list slices.
print(balanced_sum([1, 2, 3, 4]))
print(balanced_sum([1, 2, 3, 4, 5]))
print(balanced_sum([]))  # A separate empty-interval case.
```

Code 2.10: Even, odd, and empty lengths

```
10
15
0
```

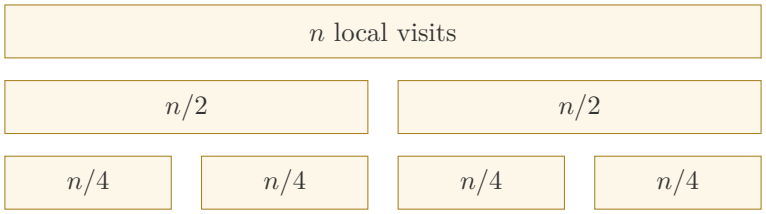
For $n \geq 1$, the algorithm reads n leaf values, performs $n - 1$ internal additions, and makes $2n - 1$ calls. The deepest active chain contains $\lceil \log_2 n \rceil + 1$ frames. Thus it has linear time and logarithmic auxiliary stack storage under the stated model. Empty input is a separate one-call case.

The bounds must describe a valid interval of the supplied list. The function is not a general interval-validation utility. Also, balancing the computation is not a promise that this Python function outperforms a simple loop, nor a universal guarantee of identical floating-point rounding.

2.10. Full local scans at every tree level

Lecture-slide alignment: Week 11 source PDF, pp. 42–43.

Suppose both half-problems are processed, but each internal call also scans its entire local input. At level zero, the scan visits n positions. At the next level, the two scans each visit $n/2$ positions, still totaling n. At the following level, four scans of $n/4$ again total n.



Total local work at each internal level: n visits.

Figure 2.3.: Two disjoint half-problems with a full local scan: summing across a level gives n work, repeated over logarithmically many levels. Source PDF p. 42; redrawn.

For power-of-two n, there are $\log_2 n$ internal levels. Adding the leaves gives

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

The source companion includes a direct counter for this conceptual structure. It is *not* a sorting algorithm.

```python
# Source teaching implementation; see the surrounding contract.
def scan_tree_work(n):
    """Conceptual work count for two half-problems plus a full scan.

    Precondition: n is a positive power of two. A leaf costs one visit.
    This is a counting model, not a sorting algorithm.
    """
    if n == 1:
        return 1
    visits_here = 0
    for _ in range(n):
        visits_here += 1  # A full local scan at this internal node.
    return (visits_here + scan_tree_work(n // 2)
            + scan_tree_work(n // 2))
```

Code 2.11: Earlier source-module counter supporting the full-scan tree

```python
# ENG 101 | Week 11 | A full local scan at every tree level
# Worked completion using earlier source module: p. 42.
from week11_models import scan_tree_work
# This counter requires a positive power-of-two size.
for n in [1, 2, 4, 8]:
    print(n, scan_tree_work(n))
```

Code 2.12: Worked counting calls for positive powers of two

```
1 1
2 4
4 12
8 32
```

Since a leaf costs one visit, the exact counter is $n(\log_2 n + 1)$ for the allowed power-of-two sizes. The helper does not validate that precondition; use only the documented inputs. The general lesson is to explain work per level rather than count levels alone.

Recursive pattern	Local work	Time growth
One call on $n - 1$	Constant	$\Theta(n)$
One call on $n/2$	Constant	$\Theta(\log n)$
Two disjoint calls on $n/2$	Constant	$\Theta(n)$
Two disjoint calls on $n/2$	Linear in local size	$\Theta(n \log n)$
One call on $n - 1$	Linear copying	$\Theta(n^2)$

The rows assume suitable base cases and the stated representation. Copying before calls, costly comparison operations, or overlapping subproblems may change the analysis.

2.11. Fibonacci: the subproblems overlap

Lecture-slide alignment: Week 11 source PDF, pp. 44–46.

The slides retain the Week 6 convention

$$F(0) = F(1) = 1, \qquad F(n) = F(n-1) + F(n-2) \quad (n \geq 2).$$

Do not silently substitute the alternative $F(0) = 0$ convention when predicting results.

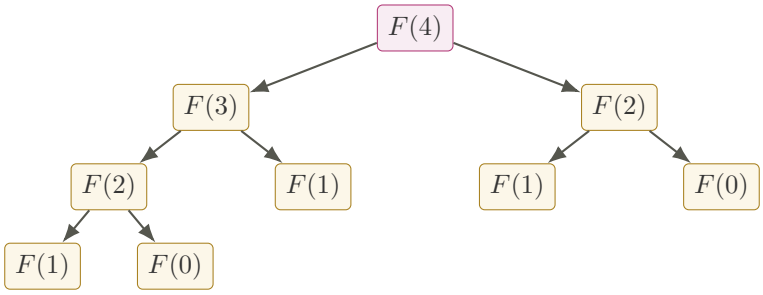


Figure 2.4.: All nine invocations in the naive $F(4)$ calculation, using $F(0) = F(1) = 1$. The repeated $F(2)$ nodes are recomputed subproblems, not a shared cached result. Source PDF p. 44; redrawn.

The full $F(4)$ tree contains nine calls and produces the value five. Identically labelled nodes are separate invocations in the naive implementation. Each repeats the work required to solve the same subproblem.

```python
# Source teaching implementation; see the surrounding contract.
def fibonacci_counted(n):
    """Return (F(n), calls), with F(0)=F(1)=1; use small n >= 0."""
    if n <= 1:
        return 1, 1  # This base-case call counts too.
    left, left_calls = fibonacci_counted(n - 1)
    right, right_calls = fibonacci_counted(n - 2)
    return left + right, 1 + left_calls + right_calls
```

Code 2.13: Source function: return both the Fibonacci value and total calls

```python
# ENG 101 | Week 11 | Repeated subproblems and total calls
# Source demonstration: pp. 44-45.
from week11_models import fibonacci_counted
# Keep the course convention F(0) = F(1) = 1.
for n in [4, 5, 10, 20]:
    value, calls = fibonacci_counted(n)
    print(n, value, calls)
```

Code 2.14: Values and call counts from the source examples

```
4 5 9
5 8 15
10 89 177
20 10946 21891
```

Let $C(n)$ count all calls, including base cases. The recurrence is

$$C(0) = C(1) = 1, \qquad C(n) = 1 + C(n-1) + C(n-2).$$

A useful worked consequence is $C(n) = 2F(n) - 1$: it holds at both bases and substitution preserves it for every later index. This explains why $F(20) = 10\,946$ corresponds to 21 891 calls.

An exponential upper bound is not a tight base

The slides describe exponential work and give $O(2^n)$ as an upper bound, not as an exact or tight call formula. The largest chain still has linear depth, so exponential total calls do not imply exponential simultaneous stack frames. The exact count recurrence above is enough to compare the supplied small cases without changing the source convention.

```python
# Source teaching implementation; see the surrounding contract.
def fibonacci_iterative(n):
    """Compute the Week 6 sequence for a nonnegative integer n."""
    if n <= 1:
        return 1
    previous, current = 1, 1
    for _ in range(2, n + 1):
        # Store only two sequence values; their bit lengths can grow.
        previous, current = current, previous + current
    return current
```

Code 2.15: Source alternative: retain two sequence values

```python
# ENG 101 | Week 11 | Keep two sequence values
# Source demonstration: p. 46.
from week11_models import fibonacci_iterative
# The same sequence is generated without an exponential call tree.
for n in [0, 1, 5, 10, 20]:
    print(n, fibonacci_iterative(n))
```

Code 2.16: Check bases and familiar sequence values

```
0 1
1 1
5 8
10 89
20 10946
```

For $n \geq 2$, the loop performs $n - 1$ recurrence additions. At $n = 20$, this is 19 additions rather than the large tree of calls. Tuple assignment evaluates the old values on the right before rebinding both names, so the previous value is not lost prematurely.

The iterative method has $\Theta(n)$ arithmetic operations and a constant number of retained sequence variables. This is *not* a full bit-complexity claim: the numerical values grow, so the storage and arithmetic cost of arbitrary-precision integers can grow too.

2.12. Memoization: make reused subproblems explicit

Lecture-slide alignment: Week 11 source PDF, pp. 47–49.

A cache stores a solved result under a key representing that subproblem. For this sequence, the index n is sufficient because the recurrence and base values are fixed. The initial cache is `\{0: 1, 1: 1\}`.

```python
# Source teaching implementation; see the surrounding contract.
def fibonacci_cached(n, cache):
    """Use a correct cache initially containing {0: 1, 1: 1}; n >= 0."""
    if n in cache:
        return cache[n]  # A previously solved subproblem.
    # Store the result once, then reuse it in later calls.
    cache[n] = (fibonacci_cached(n - 1, cache)
                + fibonacci_cached(n - 2, cache))
    return cache[n]
```

Code 2.17: Source function: inspect the cache before making new recursive calls

The first branch returns an existing result. Otherwise the two smaller values are requested, their sum is stored under `n`, and the newly stored result is returned. Because the same dictionary object is passed throughout, later calls can see results established by earlier completed calls.

```python
# ENG 101 | Week 11 | An explicit, correct cache
# Source demonstration: pp. 47-50.
from week11_models import fibonacci_cached
# Start with only the two base cases: this is a fresh-cache request.
cache = {0: 1, 1: 1}
print(fibonacci_cached(10, cache), len(cache))
before = cache.copy()
print(fibonacci_cached(10, cache))  # Now the requested answer exists.
print(cache == before)  # A warm hit adds no new cached subproblem.
```

Code 2.18: A fresh request followed by a warm hit

```
89 11
89
True
```

After the fresh request for $n = 10$, the dictionary contains the results for indices 0 through 10: 11 entries. Repeating the request uses an already available value and adds no new entries.

Method	Work model	Retained working state
Naive Fibonacci	Exponential calls	Linear-depth stack.
Fresh-cache Fibonacci	Expected $\Theta(n)$ arithmetic/hash operations	Linear cache entries and linear-depth stack.
Iterative Fibonacci	$\Theta(n)$ arithmetic operations	Two sequence values.

For one sequence value, iteration is simpler here. A cache is valuable when a larger computation needs overlapping solved states. The table assumes a correct initial cache, valid nonnegative indices, bounded-cost arithmetic, and ordinary hashing. An arbitrary user-modified cache can return an incorrect result immediately.

2.13. The cache key must identify the same problem

Lecture-slide alignment: Week 11 source PDF, p. 50.

A stress result depends on force, area, the unit convention, and the physical model. A cache indexed only by force can incorrectly reuse a value after the area changes. This is a semantic

defect: the lookup may be computationally correct but the stored result no longer belongs to the requested problem.

Document all relevant inputs to the key, avoid hidden changing global parameters, and state when cached results must be cleared or rebuilt. Passing the cache explicitly keeps its state visible. A shared mutable default dictionary would introduce an additional lifetime and sharing decision that the caller may not expect.

Engineering check

A faster warm-cache hit is not a fair substitute for a fresh calculation when the intended task starts with no solved results. Report the preparation work and retained storage separately. In a real process, include the cost of keeping cached data valid.

2.14. Worked activity, practical limits, and exit-ticket answers

Lecture-slide alignment: Week 11 source PDF, pp. 51–53.

Activity: defend each time and space claim. Index-recursive summation has one bounded-cost step per input and a linear chain. Tail slicing adds a triangular number of copied references, retained through the waiting frames. Balanced index summation visits every leaf but avoids copies and has logarithmic depth. Fresh-cache Fibonacci computes each new index once under ordinary hashing, but keeps both the cache and a linear first-descent stack. Each explanation identifies the cost that a superficial label would omit.

A function may terminate mathematically yet exceed Python's permitted call depth before reaching its base case. Keep recursive demonstrations small. Changing the recursion limit does not repair a poor choice of data-processing structure or prove adequate memory. Prefer an iterative form when the task is naturally a long sequence of updates. The optional source reference to `sys.getrecursionlimit()` reports the interpreter's configured limit; it does not establish a universal safe input size.

Exit question 1: How can one child call per level still give quadratic time? A linear-cost slice or scan on each successively smaller problem produces a triangular cumulative workload.

Exit question 2: Why is logarithmic depth not logarithmic total time? The algorithm may visit many nodes at a level or perform a growing amount of local work. The complete balanced tree already has linear total calls.

Exit question 3: What is excluded when timing only a warm cache hit? The calculation that first filled the cache, its construction and retained storage, and any later invalidation or maintenance.

Exit question 4: What must be rechecked after replacing recursion? Preserve the base cases, required value or record ordering, units, input non-mutation, documented errors, and a suitable numerical accuracy criterion. A new implementation can have a better count and still violate one of these promises.

3 Scaling Experiments and Engineering Choices

Lecture-slide alignment: Week 11 source PDF, pp. 54–78.

Learning outcomes

By the end of this unit, you should be able to derive a moving-window cost in both n and w; verify equivalent output contracts; interpret doubling ratios and empirical order; distinguish recorded measurements from count models; and explain time, memory, accuracy, and end-to-end trade-offs.

3.1. Predict the scaling before measuring it

Lecture-slide alignment: Week 11 source PDF, pp. 54–57.

For a declared count function $C(n)$, doubling the problem size produces the exact ratio $C(2n)/C(n)$. The table concerns the displayed functions, not every possible algorithm sharing the same Big-O upper bound.

Pure count model	Exact doubling ratio, in its domain
1	1
$\log_2 n$	$1 + 1/\log_2 n$, for $n > 1$
n	2
$n \log_2 n$	$2(1 + 1/\log_2 n)$, for $n > 1$
n^2	4
n^3	8
2^n	2^n

For the logarithmic row, $\log_2(2n) = 1 + \log_2 n$; division by $\log_2 n$ gives the result. For the exponential row, $2^{2n}/2^n = 2^n$. Doubling n does not merely double an exponential count.

The analogous measured ratio is

$$R(n) = \frac{t(2n)}{t(n)}, \qquad \widehat{p}(n) = \frac{\log R(n)}{\log 2}.$$

For the exact power model $t(n) = an^p$, the ratio is 2^p, so $\widehat{p} = p$. Real measurements can contain fixed overhead, different input paths, allocation costs, cache effects, and background activity. A finite-size estimate therefore need not equal the asymptotic exponent.

Common mistake

A measured ratio close to four is supporting evidence for quadratic-like behavior over those cases, not a proof of $\Theta(n^2)$. Conversely, a noisy ratio far from four does not erase a justified quadratic operation count. Analyze the implementation and the experiment separately, then explain their agreement or disagreement.

3.2. Specify every full-window mean

Lecture-slide alignment: Week 11 source PDF, p. 58.

The engineering case asks for all averages of consecutive full windows in a temperature record. If n readings are $T_0, \ldots, T_{n-1}$ and the window contains w readings, then

$$\overline{T}_i = \frac{1}{w} \sum_{j=i}^{i+w-1} T_j, \qquad i = 0, \ldots, n - w.$$

Exactly $M = n - w + 1$ results are required. The output has the same temperature unit as the inputs. It is ordered by increasing starting position; the program does not pad incomplete windows or reorder the record.

Function contract

Inputs are finite numeric readings and an integer width satisfying $1 \leq w \leq n$. Return *every* full-window mean in time order, leave the input unchanged, and reject an invalid width. The source implementation assumes valid readings; it does not independently validate every element. The numerical suitability of the computation still needs testing.

For `[20, 22, 24, 26, 28]` and width three, the windows are `[20, 22, 24]`, `[22, 24, 26]`, and `[24, 26, 28]`. Their means are 22, 24, and 26 degrees Celsius. This hand calculation provides an expectation independent of either implementation.

Engineering check

A moving average can conceal a short sharp event. Computing the stipulated averages correctly does not establish that smoothing is appropriate for the physical decision or that each individual reading meets a limit. The case studies the cost of a defined output, not the design of a validated signal-processing system.

3.3. Direct calculation: recompute every window

Lecture-slide alignment: Week 11 source PDF, pp. 59–60.

The earlier source module supplies the helper imported or referenced by the slides:

```python
# Source teaching implementation; see the surrounding contract.
def check_width(values, width):
    """Validate width; callers separately ensure finite numeric data."""
    if isinstance(width, bool) or not isinstance(width, int):
        raise TypeError("width must be an integer, not a Boolean")
    if not 1 <= width <= len(values):
        raise ValueError("require 1 <= width <= number of readings")
```

Code 3.1: Recovered source helper: width validation without a data-validation scan

Booleans must be rejected explicitly because Python treats `bool` as a subclass of `int`. The helper first distinguishes the type contract, then checks the inclusive valid width interval. It raises `TypeError` for a Boolean or noninteger width, and `ValueError` for an integer outside the permitted range.

```python
# Source teaching implementation; see the surrounding contract.
def moving_mean_direct(values, width):
    """Return every full-window mean; input is not modified."""
    check_width(values, width)
    means = []
    for start in range(len(values) - width + 1):
        total = 0.0
        for index in range(start, start + width):
            total += values[index]  # Revisit overlapping readings.
        means.append(total / width)
    return means
```

Code 3.2: Source function: revisit the overlapping readings

The outer loop runs over starting positions 0 through $n-w$. The inner loop covers `start` through `start + width - 1`; its stop is excluded. The subtotal starts at zero for each window, and the completed mean is appended once.

```python
# ENG 101 | Week 11 | Sum each full window independently
# Source demonstration: pp. 58-60.
from week11_models import moving_mean_direct
# Temperatures in deg C; each window contains three readings.
values = [20.0, 22.0, 24.0, 26.0, 28.0]
print(moving_mean_direct(values, 3))
print(values)  # The original time order is retained.
```

Code 3.3: A complete small client call

```
[22.0, 24.0, 26.0]
[20.0, 22.0, 24.0, 26.0, 28.0]
```

Every one of $n - w + 1$ windows performs exactly w selected additions, so

$$C_{\text{direct}}(n, w) = w(n - w + 1).$$

This count excludes divisions, loop control, width checks, and output writes. Under the bounded-cost model those additional operations do not change its growth order over valid widths, because $w \geq 1$ and each output already requires at least one counted addition.

How w changes with n	**Interpretation of the exact count**
Fixed positive w	Linear in n as the record grows.
$w = n/4$, with n divisible by four	A quadratic term $3n^2/16$, plus a linear term $n/4$.
$w = n$	One window; exactly n additions.
$w = 1$	n singleton windows; exactly n additions.

Thus the mere presence of two loops does not imply quadratic work for every scaling path. Writing only $O(nw)$ is a valid upper-bound summary here, but the exact count exposes the one-window case $w = n$ much more clearly.

3.4. Reuse the overlap: derive the running update

Lecture-slide alignment: Week 11 source PDF, pp. 61–63.

Let S_i be the subtotal of the window beginning at position i. The next window discards T_i and introduces T_{i+w}. Algebraically,

$$S_{i+1} = S_i - T_i + T_{i+w}.$$

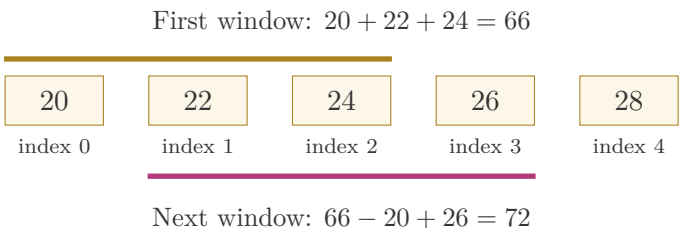


Figure 3.1.: Adjacent full windows share their interior readings. The displayed update is an algebraic identity; numerical adequacy is tested separately. Source PDF p. 61; redrawn.

For the small example, $S_0 = 20 + 22 + 24 = 66$. Moving right gives $S_1 = 66 - 20 + 26 = 72$, and then $S_2 = 72 - 22 + 28 = 78$. Division by three produces the same required means.

```python
# Source teaching implementation; see the surrounding contract.
def moving_mean_running(values, width):
    """Reuse a window total; reassess floating-point error for real data."""
    check_width(values, width)
    total = 0.0
    for index in range(width):
        total += values[index]  # Build the first window only once.
    means = [total / width]
    for index in range(width, len(values)):
        total -= values[index - width]  # Remove the outgoing value.
        total += values[index]          # Add the incoming value.
        means.append(total / width)
    return means
```

Code 3.4: Source function: one initial sum followed by two updates per shift

The first loop builds the first subtotal using exactly w additions. The initial result list contains its mean. The second loop begins at index w: this is the first new incoming position, not a new window start. The outgoing position is `index - width`. Subtraction removes that value; addition introduces the new one. One mean is appended after both updates.

```python
# ENG 101 | Week 11 | Reuse the shared window interior
# Source demonstration: pp. 61-63.
from week11_models import moving_mean_running
# First subtotal: 66; next: 66 - 20 + 26 = 72.
values = [20.0, 22.0, 24.0, 26.0, 28.0]
print(moving_mean_running(values, 3))
print(values)
```

Code 3.5: Small client call to the running method

```
[22.0, 24.0, 26.0]
[20.0, 22.0, 24.0, 26.0, 28.0]
```

Stage	Outgoing index	Incoming index	Subtotal	Mean
Initial	—	0, 1, 2	66	22
Shift 1	0	3	72	24
Shift 2	1	4	78	26

The second loop has $n-w$ passes, each containing one subtraction and one addition. The selected arithmetic count is consequently

$$C_{\text{running}}(n, w) = w + 2(n - w) = 2n - w.$$

Since $1 \le w \le n$, it lies between n and $2n - 1$, giving linear arithmetic work in n across all valid widths.

n	$w = n/4$	Direct additions	Running additions/subtractions
8	2	14	14
16	4	52	28
32	8	200	56
64	16	784	112

At the smallest listed case the selected counts are equal. For $w = 1$, the direct method uses n additions, whereas the running method uses $2n - 1$ updates; both are linear. Better large-size growth does not guarantee a smaller count in every small or special case. These count ratios are not measured speedups.

3.5. Test values, output length, boundaries, and non-mutation

Lecture-slide alignment: Week 11 source PDF, p. 64.

```python
# ENG 101 | Week 11 | Known answers and valid-width boundaries
# Source demonstration: p. 64.
from week11_models import moving_mean_direct, moving_mean_running
# These chosen values have exact expected outputs in binary floats.
values = [20.0, 22.0, 24.0, 26.0, 28.0]
original = values.copy()
for method in [moving_mean_direct, moving_mean_running]:
    assert method(values, 3) == [22.0, 24.0, 26.0]
    assert method(values, 1) == values
    assert method(values, 5) == [24.0]
    assert values == original  # Both methods must be non-mutating.
print("Moving-mean checks passed")
```

Code 3.6: Source known-answer and boundary checks

```
Moving-mean checks passed
```

The width-one case requires a result equal to the original readings, but not mutation of them. The full-length window requires one result, 24. The width-three case verifies all intermediate windows rather than only the first or last. Comparing with a saved input copy checks another independent promise.

```python
# ENG 101 | Week 11 | Rejected widths are part of the contract
# Worked completion of invalid-input tests: pp. 59, 64.
from week11_models import moving_mean_direct
# Demonstrate the checks; these exceptions are caught deliberately.
for width in [0, 6, 2.5, True]:
    try:
        moving_mean_direct([20.0] * 5, width)
    except (TypeError, ValueError) as error:
        print(repr(width), type(error).__name__)
# Empty data cannot supply a valid full window of positive width.
try:
    moving_mean_direct([], 1)
except ValueError as error:
    print("empty:", type(error).__name__)
```

Code 3.7: Worked completion of the source's invalid-width test list

```
0 ValueError
6 ValueError
2.5 TypeError
True TypeError
empty: ValueError
```

A complete test plan checks both implementations on each rejected case. The printed exception-type demonstration uses the direct method to avoid duplicating a long output. The supplied test suite checks both.

A width check is not a complete numerical input check

`check_width()` does not scan the values, verify physical units, or ensure that intermediate arithmetic cannot overflow. Those are separate preconditions and numerical questions. Adding a full finite-data validation pass would change the measured region if performed inside every call, even though the running method would remain linear in the bounded-cost model.

3.6. A faster algebraic update can lose required information

Lecture-slide alignment: Week 11 source PDF, p. 65.

```python
# ENG 101 | Week 11 | A numerical counterexample, not an indexing defect
# Intentional numerical counterexample: p. 65.
from week11_models import moving_mean_direct, moving_mean_running
# Large-plus-small cancellation exposes lost floating-point information.
values = [1e16, 1.0, 1.0, 1.0]
print(moving_mean_direct(values, 2))
print(moving_mean_running(values, 2))
```

Code 3.8: The source floating-point stress test

```
[5000000000000000.0, 1.0, 1.0]
[5000000000000000.0, 0.5, 0.5]
```

The first output is from the direct method; the second is from the running method. Their first large mean is the same in this ordinary binary-float demonstration, but their later small means are not.

Trace the critical subtotal. The first window contains `1e16` and `1.0`. In the float representation used by the source example, their sum rounds to `1e16`: the small contribution is lost. When the large value leaves the running window, subtraction gives zero rather than the mathematically required remaining one. Adding the incoming one therefore leaves subtotal one and mean 0.5. The next shift preserves the same missing contribution.

The direct method starts each later subtotal afresh, so it computes $1 + 1 = 2$ and mean one for those windows. This is not evidence that the direct algorithm is always numerically reliable; it is a counterexample to accepting the running method unconditionally.

Engineering check

The outgoing and incoming indices are correct. The problem is not repaired by renaming a variable or changing a loop boundary. Reassess the numerical representation, calculation, or recomputation strategy when the error exceeds the application's accuracy requirement. Do not hide the failure by loosening a tolerance until the test passes.

The slides establish a limitation, not a fully stable production replacement. No such guarantee is added here. Retain this test separately from the ordinary small-value correctness cases.

3.7. Work, workspace, and returned output

Lecture-slide alignment: Week 11 source PDF, p. 66.

Resource	Direct windows	Running subtotal
Selected updates	$w(n - w + 1)$	$2n - w$
Extra scalar workspace	Constant	Constant
Returned mean values	$M = n - w + 1$	$M = n - w + 1$
Supplied record	Preserved	Preserved

Both implementations retain a result list whose size is proportional to M, not always n: when $w = n$, $M = 1$. Apart from the supplied input and the required result, only a fixed number of scalar variables is retained. That is the convention behind the source's constant-workspace statement.

Keeping only the final mean would reduce the output requirement. It could be a legitimate redesign for a different problem, but it is not a like-for-like improvement to the stipulated all-means function.

3.8. Plan the timing experiment around a fixed question

Lecture-slide alignment: Week 11 source PDF, pp. 67–68.

A fair comparison uses equivalent inputs and the same width, required outputs, numerical acceptance criteria, and failure behavior. Fixture construction, correctness assertions, counters, printing, and file I/O should remain outside the measured callable when the declared scope is calculation-only.

```python
# ENG 101 | Week 11 | A local timing observation
# Machine-dependent timing demonstration: p. 68.
from timeit import repeat
from week11_models import moving_mean_running
# Set up inputs outside the callable measured by timeit.
values = [20.0 + (i % 21) * 0.5 for i in range(256)]
width = 64

def task():
    # Create and return a new result on every timed invocation.
    return moving_mean_running(values, width)

# This source demonstration has five trials of 100 calls each.
trials = repeat(task, repeat=5, number=100)
print(min(trials) / 100)  # Fastest observed seconds per call; variable.
```

Code 3.9: The source five-trial timing demonstration, with comments

The program prints a machine-dependent duration in seconds per call; there is no fixed expected numerical output.

`repeat(task, repeat=5, number=100)` returns five batch durations. Each duration covers 100 calls, so division by 100 converts the fastest batch duration into a per-call observation. It is not correct to divide by the number of trials instead.

The short demonstration and the source dataset use different protocols

The displayed short program uses five trials and 100 calls per trial. The historical scaling dataset uses seven trials per case and adaptive batch lengths targeting approximately 0.03 seconds. The distinction is explicit in the slides and is retained here. Neither set of durations should be treated as fixed expected output for a student machine.

The recorded scaling experiment uses

$$T_i = 20.0 + 0.5(i \bmod 21), \qquad w = n/4,$$

with $n \in \{128, 256, 512, 1024, 2048\}$. At these sizes and power-of-two widths, the simple half-degree fixture gives exactly agreeing outputs for the two methods. Agreement is checked before timing, and data preservation is also checked. The choice makes the timing comparison clean; it does not remove the numerical limitation demonstrated previously.

The measured region includes the call, width validation, calculation, creation of the result list, and release of that result. By default, `timeit` disables cyclic garbage collection within a batch. That may not represent every allocation-heavy application's environment; the slides therefore require a representative end-to-end check before deployment claims. The Python documentation is supplied as a technical reference, not a new source of course content [2].

3.9. Read the original measurements as a recorded experiment

Lecture-slide alignment: Week 11 source PDF, pp. 69–70.

Historical source dataset, preserved unchanged

The earlier benchmark record identifies CPython 3.13.5, a 64-bit Linux x86_64 environment, and `timeit.default_timer` (`perf_counter`). Its recorded timestamp is 14 September 2026, 05:08:56 UTC. There are seven normalized trials per method and size. These are source observations, not freshly generated values for this document.

n	w	Direct (μs)	Running (μs)	Direct ratio	Running ratio
128	32	67.802	7.078	—	—
256	64	242.628	13.925	3.58	1.97
512	128	1136.754	30.763	4.69	2.21
1024	256	4920.444	65.544	4.33	2.13
2048	512	20054.962	136.707	4.08	2.09

Table 3.1.: Historical source timings with ratios calculated from the unrounded recovered data. Rounded display values do not replace the raw trial record. Source PDF p. 69.

The direct and running columns give the minimum batch duration divided by its call count, converted to microseconds. Each ratio divides a row's time by the preceding row's time for the same implementation. The first row has no preceding size.

For example, the direct ratio from 256 to 512 readings is larger than four, whereas the running ratio is closer to two. The exact count model predicts the broad difference; measured details can reflect interpreter behavior, allocation, and finite-size effects. A perfect integer ratio is neither expected nor required for this experiment.

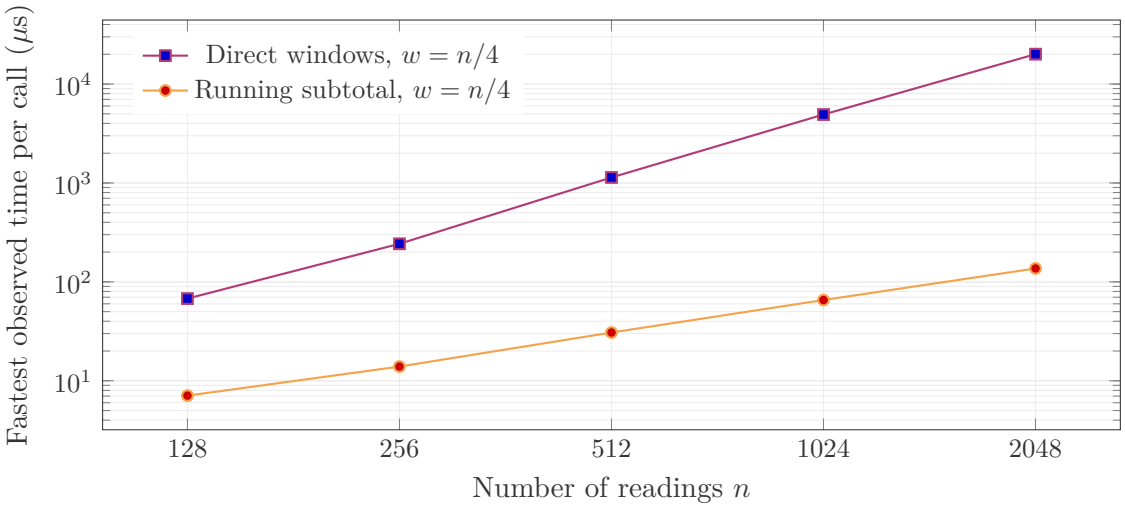


Figure 3.2.: The original source benchmark: minimum of seven batch-normalized observations for each method and size. Lines join measured points; they are not fitted laws. Source PDF p. 70 and recovered `timing_summary.csv`.

Both axes are logarithmic. The connecting segments guide the eye between observations; they are not fitted complexity laws. The plotted numbers are read from the original CSV. No textbook or analytic curve is substituted for a measured series.

3.10. What the timing supports, and what it does not

Lecture-slide alignment: Week 11 source PDF, p. 71.

A defensible conclusion is: for this workload and these tested sizes, the running-subtotal method used substantially less observed time, consistent with its lower selected arithmetic count. That conclusion retains the width policy, input fixture, and measurement environment.

It does not establish a universal speedup, a real-time deadline, or numerical reliability on every input. It also does not turn finite timing samples into a proof of asymptotic growth. If w is fixed while n grows, the direct implementation is also linear in n. Omitting the width from the report would hide the workload that made the original experiment quadratic.

Key idea

Use code-based counts to explain why work grows. Use timing to assess a particular implementation over a declared workload. Use accuracy and contract tests to decide whether its output remains acceptable. These are complementary forms of evidence.

3.11. Operation budgets and multiple channels

Lecture-slide alignment: Week 11 source PDF, pp. 72–73.

For $n = 2000$ readings and $w = 500$, the output length is 1501. The selected counts are

$$C_{\text{direct}} = 500(1501) = 750\,500, \qquad C_{\text{running}} = 4000 - 500 = 3500.$$

Under the source's *illustrative* budget of 100 000 arithmetic updates, only the running method fits. The budget excludes operations outside the selected count and has no unit of seconds. It must not be presented as a measured deadline.

```python
# ENG 101 | Week 11 | A selected-operation budget is not a deadline
# Worked arithmetic illustration: p. 72.
# Calculate only the arithmetic counts defined in the source.
n, width = 2000, 500
outputs = n - width + 1
direct = width * outputs
running = 2 * n - width
budget = 100_000
print("Outputs:", outputs)
print("Direct:", direct, direct <= budget)
print("Running:", running, running <= budget)
```

Code 3.10: Worked arithmetic for the hypothetical count budget

```
Outputs: 1501
Direct: 750500 False
Running: 3500 True
```

For m independent channels, each with n readings and the same width, the sequential arithmetic models are

$$m\,w(n - w + 1) \quad \text{and} \quad m(2n - w).$$

Retaining every result requires $m(n - w + 1)$ output values. If $m = n$ and $w = n/4$, direct work becomes cubic in n, running work becomes quadratic, and the required output is still quadratic. No parallel execution or automatic sharing between channels is assumed.

3.12. End-to-end speedup includes unchanged stages

Lecture-slide alignment: Week 11 source PDF, p. 74.

The source uses explicitly hypothetical stage durations:

Stage	Before (s)	After (s)
Read and validate	0.60	0.60
Calculate	0.30	0.03
Write the report	0.10	0.10
Total	1.00	0.73

The calculation is ten times faster, but total speedup is only $1.00/0.73 \approx 1.37$. The unchanged stages still consume 0.70 seconds. It is therefore important to improve the stage that limits the actual workflow, not simply the easiest loop to optimize.

```
# ENG 101 | Week 11 | Hypothetical end-to-end timing budget
# Worked hypothetical calculation: p. 74.
# All durations here are assumed examples, not measured data.
read_s, calculate_s, report_s = 0.60, 0.30, 0.10
before_s = read_s + calculate_s + report_s
after_s = read_s + calculate_s / 10 + report_s
print(f"Before: {before_s:.2f} s")
print(f"After: {after_s:.2f} s")
print(f"Overall speedup: {before_s / after_s:.2f}")
```

Code 3.11: Worked arithmetic using hypothetical durations, not the benchmark data

```
Before: 1.00 s
After: 0.73 s
Overall speedup: 1.37
```

3.13. Review an improvement, prepare the lab, and report evidence

Lecture-slide alignment: Week 11 source PDF, pp. 75–77.

The peer-review proposal keeps only the last moving mean, removes width checks, and reports a large speedup. It is not an acceptable comparison to the original contract. It changes the output, weakens documented failure behavior, and may omit numerical evidence.

Worked review. Compare implementations returning the same $n - w + 1$ means on equivalent input with the same width and validation scope. Retain the known-answer and boundary cases, data-preservation tests, and cancellation stress test. Report the measured region and all relevant growing dimensions. A speed claim alone cannot justify changing the result or ignoring rejected inputs.

Laboratory 4 preparation, not a new assignment

The source lists Laboratory 4 and suggests practicing an (n, w) cost model, known-answer checks, a small scaling table, and a time–space–accuracy discussion. It does not specify a new compulsory dataset, deadline, mark allocation, or submission format. The separate Moodle brief remains authoritative for graded requirements and permitted assistance.

A defensible efficiency report answers six questions in ordinary sentences: what result and accuracy are required; which input dimensions and cases are analyzed; what work is justified; what preparation, output, and memory costs are included; which tests continue to pass; and what was measured on which environment with what limitations. This connects the efficiency unit to function contracts, regression tests, and state invariants from earlier weeks.

3.14. Exit-ticket answers and the next unit

Lecture-slide alignment: Week 11 source PDF, p. 78; p. 87.

For $n = 12$ and $w = 3$, the required output has $12 - 3 + 1 = 10$ means. The direct method performs $3 \times 10 = 30$ selected additions. The running method performs $3 + 2(12 - 3) = 21$ additions/subtractions. These counts are independently calculable before executing either program.

Timing alone does not determine the better engineering choice: numerical accuracy, output equivalence, valid-domain handling, data preservation, memory, and the intended workload also matter. The next unit applies the same discipline to search and sorting implementations.

Section summary

A faster program is useful only when it still solves the specified problem. Expose hidden work, justify growth, measure the right task, and keep the engineering result trustworthy.

4 Optional Extensions and Reference

Lecture-slide alignment: Week 11 source PDF, pp. 79–86.

This chapter retains the source deck's optional material as optional. It supports the main explanations without creating additional prerequisites or assessment requirements. The familiar-container cost table is already discussed in Section 1.3; its qualifications still apply.

4.1. A bound for the toy resizing strategy

Lecture-slide alignment: Week 11 source PDF, p. 80.

The doubling-capacity picture can be justified with a geometric sum. Across $n \geq 1$ appends, resizing copies existing capacities $1, 2, 4, \ldots, 2^k$, where the last copied capacity is smaller than n. Thus

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1 < 2n.$$

Every append additionally writes one new value. Across the whole sequence,

$$C_{\text{total}}(n) < 2n + n = 3n.$$

Dividing by n bounds the average selected work per append by three model units. That is the amortized argument: the expensive resizes are accounted for rather than ignored.

For exactly $n = 8$, the earlier demonstration has seven copies plus eight writes, giving 15. The general upper bound is deliberately not claimed to be that exact total for every n.

Technical clarification

The proof is about the source's explicit toy strategy. CPython uses its own proportional overallocation policy. Neither the model's capacities nor its bars should be reported as actual allocation measurements. No probabilistic input distribution was assumed in deriving the sequence bound.

4.2. Factorial growth: enumerating all orderings

Lecture-slide alignment: Week 11 source PDF, p. 81.

```python
# ENG 101 | Week 11 | Visit every ordering, or calculate one number
# Optional source demonstration: p. 81.
from itertools import permutations
from math import factorial
# Keep four distinct labels: enumerating all orderings grows rapidly.
visits = 0
for order in permutations(range(4)):
    visits += 1  # One counted visit for each generated tuple.
print(visits)
print(factorial(4))  # Same number, a different computational task.
```

Code 4.1: Source optional example: 24 generated orderings and one factorial value

```
24
24
```

The loop visits every ordering of four distinct labels and increments a counter for each generated tuple. The subsequent factorial call returns one integer. Both print 24, but they solve different computational tasks.

For n distinct labels there are $n!$ complete orderings. Merely counting one visit per ordering gives $n!$ visits. Explicitly storing every tuple of length n requires $\Theta(n\,n!)$ element slots. Constructing or inspecting every element of each tuple can also add a factor of n to an analysis that counts more than tuple visits.

The demonstration does not store all tuples in a list, and the source does not assert a detailed peak-memory model for the iterator itself. Keep those distinctions separate. Calculating $n!$ through multiplication does not enumerate all permutations. Always preserve the stated output contract before assigning factorial growth.

4.3. Preparation before repeated searches

Lecture-slide alignment: Week 11 source PDF, p. 82.

The source previews a unit-coefficient comparison, not a complete implementation. For q queries on n records, let the simplified scan cost be qn. Preparing an ordered index and then making logarithmic queries gives the model

$$n \log_2 n + q \log_2 n.$$

Under these explicitly chosen coefficients, the second expression is smaller when

$$\begin{aligned} n \log_2 n + q \log_2 n &< qn, \\ n \log_2 n &< q(n - \log_2 n), \\ q &> \frac{n \log_2 n}{n - \log_2 n}, \end{aligned}$$

provided the denominator is positive.

```python
# ENG 101 | Week 11 | A unit-coefficient comparison only
# Worked optional model calculation: p. 82.
from math import log2
# This model uses equal coefficients; it does not predict seconds.
n = 16
threshold = n * log2(n) / (n - log2(n))
print(f"Required q > {threshold:.3f}")
for queries in [5, 6]:
    scan_work = queries * n
    index_work = n * log2(n) + queries * log2(n)
    print(queries, scan_work, index_work, index_work < scan_work)
```

Code 4.2: Worked optional comparison at $n = 16$

```
Required q > 5.333
5 80 84.0 False
6 96 88.0 True
```

At this size, the threshold is 16/3, so integer $q = 6$ satisfies the strict inequality and $q = 5$ does not. This is a result of the stated model, not a universal number of queries after which sorting is worthwhile.

Real crossover behavior depends on constants, data distributions, lookup and comparison costs, required metadata, and changes to the indexed data. Index maintenance must be included when the source records change. Big-O bounds alone do not determine a numerical threshold. Week 12 develops the search and sorting implementations and their preconditions.

4.4. Reproducibility without silently changing the experiment

Lecture-slide alignment: Week 11 source PDF, p. 83.

Each timed invocation should represent the same required task. Reusing an already-computed cache entry while calling the result a fresh computation changes the workload. So does allowing an in-place method to process an input already altered by the previous trial.

Retain raw trial vectors, the chosen statistic, input dimensions, and the environment. Keep counting instrumentation, plotting, and printing outside the timing scope unless they are intentionally part of an end-to-end measurement. For allocation-heavy work, assess whether disabling cyclic garbage collection changes the interpretation. A microbenchmark does not replace an application-level test.

Reproduction tool supplied with these notes

The slides refer to an earlier `tools/generate_data.py`, but that generator was not present among the recovered Week 11 files. These notes therefore supply a *new, explicitly labelled* `tools/rerun_benchmark.py` that follows the recorded input, seven-trial, normalization, and approximate batch-target protocol. It writes to `new_measurements/` by default and does not overwrite the source CSV files.

The exact pilot decisions and elapsed times from the earlier run cannot be regenerated as constants; timing is an observation. Compiling the LaTeX document reads the preserved original data only. The supplied raw-data checks verify normalization and summary calculations without imposing a brittle assertion that one function must always be faster.

4.5. Reference vocabulary and source reading

Lecture-slide alignment: Week 11 source PDF, pp. 84–86.

Chapter 6 explains the source vocabulary. The slides identify Wang's course manuscript, especially Appendix G, and Guttag's computation text as background. These notes follow the Week 11 lecture examples and timing records; Appendix A distinguishes the source material from worked additions.

Official Python documentation supplies technical references for timers, collection design, recursion limits, and floating-point arithmetic [2, 3, 4, 5]. It supports implementation qualifications, not a replacement teaching sequence.

5 Cumulative Practice and Worked Solutions

Worked extension of the slides

The following twelve questions are additional self-study exercises built from the Week 11 concepts. They are not announced examination questions or new graded tasks. Attempt each question before reading its answer. All code refers to the commented definitions already presented in these notes.

5.1. Question 1: same number of loop headers

For $n = 5$, calculate the counters in the fixed-inner and triangular warm-up loops. Give a formula and a tight growth description for each. Does the number of loop headers explain the difference?

Answer and explanation

The fixed-inner loop has $3n = 15$ visits and $\Theta(n)$ growth. The triangular loop has $0 + 1 + 2 + 3 + 4 = 10$ visits, equal to $n(n-1)/2$, and $\Theta(n^2)$ growth. The bounds, not the number of headers, determine the count. The triangular loop happens to have fewer visits at $n = 5$, despite its faster asymptotic growth.

5.2. Question 2: cumulative copies versus live storage

A list is built by repeated concatenation of one doubled value. For $n = 6$, how many result slots are populated across all concatenations? Must all six intermediate lists remain alive simultaneously?

Answer and explanation

The count is $1 + 2 + 3 + 4 + 5 + 6 = 21$. The program rebinds one result name and does not retain every earlier list. Old and new lists may coexist during an assignment, but obsolete results are not all saved. Cumulative copying is quadratic; peak result-list storage is linear. This answer depends on the actual references retained by this implementation.

5.3. Question 3: amortized is not average-case

In the toy capacity strategy, eight appends cost 15 selected units. Explain why this supports an amortized claim without assuming random inputs. Does it promise that each append costs 15/8 units?

Answer and explanation

The 15 units are the total over a prescribed sequence: eight writes plus seven resize copies. Dividing the total distributes the expensive resize work over all operations. No random input distribution appears. Individual costs are unequal; the bars are $1, 2, 3, 1, 5, 1, 1, 1$, not eight identical costs. The general geometric bound supports amortized $O(1)$ for this strategy.

5.4. Question 4: zero comparisons do not mean zero work

Three installed identifiers are all absent from a two-element approved list. How many ID equality comparisons are made? What changes if the approved list is empty? Use the following explicit instrumentation as a worked extension of the source model.

```
# ENG 101 | Week 11 | Count actual ID equality tests
# Additional worked instrumentation: p. 16.
def counted_flags(installed, approved):
    """Worked instrumentation of a list search; short string IDs only."""
    flags = []
    comparisons = 0
    for sensor_id in installed:
        found = False
        for candidate in approved:
            comparisons += 1  # Count every equality test.
            if sensor_id == candidate:
                found = True
                break  # Continue with the next installed record.
        flags.append(found)
    return flags, comparisons

# All absent: exactly q*m comparisons, yet every flag is produced.
print(counted_flags(["A", "B", "A"], ["C", "D"]))
print(counted_flags(["A", "B", "A"], []))
```

Code 5.1: Added instrumentation: make each equality comparison visible

```
([False, False, False], 6)
([False, False, False], 0)
```

Answer and explanation

The absent two-element case uses $3 \times 2 = 6$ comparisons. With no approved elements, there are zero equality comparisons, but the outer loop still creates three false flags. The output requirement therefore contributes linear work even when the selected comparison count is zero. The two printed results retain the repeated installed identifier as a separate record.

5.5. Question 5: repeated batches and stale indexes

Let $B = 4$, $q = 3$, and $m = 10$. Compare the source's unit-coefficient count models for scanning every time, rebuilding an index each batch, and building once. What must be checked before reusing the index?

Answer and explanation

For positive m, the all-absent equality-comparison model is $Bqm = 120$. The simplified expected build-plus-query model is $B(m+q) = 52$ when rebuilt each batch, versus $m+Bq = 22$ when built once. These are different model operations, not measured speedups. Reuse is valid only while the approved data and key meanings remain current. Retained flag outputs and index entries are separate storage requirements.

5.6. Question 6: removing an invariant calculation

Why is it valid to move `sum(readings) / len(readings)` outside the centering loop? Which edge case must be handled explicitly after doing so? Would the same change be valid if the loop modified `readings`?

Answer and explanation

The source function leaves the readings unchanged, so the same mean is used at every iteration. Compute it once and then form the results. The empty input requires an early return before division; the old loop skipped division automatically because it had no iterations. If the data changed inside the loop, hoisting would generally use the wrong mean. Side effects or changing external inputs would also need separate analysis.

5.7. Question 7: distinguish two halving patterns

Evaluate `geometric_work(10)` and `full_scan_each_stage(10)`. Explain each result and its growth class.

Answer and explanation

The geometric inner workloads are $10, 5, 2, 1$, totaling 18 and bounded between n and $2n$. Its growth is $\Theta(n)$. The repeated-full-scan loop has three stages, with size values $10, 5, 2$, and ten visits at each: 30 visits. Its count is $n\lfloor \log_2 n \rfloor$ and growth is $\Theta(n \log n)$. The first loop includes the size-one stage; the second stops before it.

5.8. Question 8: count recursive resources separately

For a six-element list, give the total call counts for index recursion, tail-slice recursion, and balanced index summation. Count the copied tail references and identify which version can retain them simultaneously.

Answer and explanation

Index recursion makes seven calls and six additions. Tail-slice recursion also makes seven calls and six additions, but additionally copies $5+4+3+2+1+0=15$ references. Its waiting frames retain those tails. Balanced summation makes $2n-1=11$ calls and $n-1=5$ additions, but its largest chain is only $\lceil \log_2 6 \rceil + 1 = 4$ frames. It creates no slices. More total calls here do not imply more stack memory.

5.9. Question 9: fresh and warm Fibonacci computation

Using $F(0)=F(1)=1$, find $F(6)$ and the naive total call count. How many cache entries remain after a fresh cached request for six? What does a subsequent request for six measure?

Answer and explanation

The sequence is $1, 1, 2, 3, 5, 8, 13$, so $F(6)=13$. The naive call count is $2F(6)-1=25$. Starting from the two correct base entries, the cache eventually contains indices 0 through 6: seven entries. A repeated request for six is a warm hit and does not include the earlier recursive computation or construction of those stored results. The cache must still be correct.

5.10. Question 10: compare full-window calculations

For $n=12$ and $w=4$, compute the required output length, direct additions, and running updates. How does the comparison change when $w=n$? Use the trace program to explain the outgoing and incoming indices in the smaller five-reading example.

```python
# ENG 101 | Week 11 | Trace the outgoing and incoming indices
# Additional worked trace: p. 61.
# Worked trace of the source running-window update; values are deg C.
values = [20.0, 22.0, 24.0, 26.0, 28.0]
width = 3
total = 0.0
for index in range(width):
    total += values[index]
print("first:", total, total / width)
for index in range(width, len(values)):
    outgoing = index - width  # Remove this old leftmost position.
    total -= values[outgoing]
    total += values[index]    # Include this new rightmost position.
    print(outgoing, index, total, total / width)
```

Code 5.2: Added trace: update one full-window subtotal at a time

```
first: 66.0 22.0
0 3 72.0 24.0
1 4 78.0 26.0
```

Answer and explanation

For $n = 12, w = 4$, there are nine means, $4 \times 9 = 36$ direct additions, and $24 - 4 = 20$ running additions/subtractions. For $w = n = 12$, there is one mean and both methods use 12 selected additions. In the trace, incoming indices are three and four, and outgoing indices are zero and one, obtained by subtracting width three. The shared interior is retained, not resummed.

5.11. Question 11: explain the failed numerical update

In the cancellation example, why does the running method report 0.5 for a window whose two values are both 1.0? Is the direct method now proved reliable on every finite input?

Answer and explanation

The first large subtotal rounded `1e16 + 1.0` to `1e16`. When the large term was subtracted, the lost contribution could not reappear. The new small value then produced subtotal one, not two. The running indices are correct, but the finite-precision state lacks necessary information. This one case does not prove universal accuracy for direct resummation either. The application's numerical criterion must be tested independently of speed.

5.12. Question 12: distinguish models, observations, and conclusions

A report says, "Our code is $O(n)$, so it always meets a 1 ms deadline. It is ten times faster because the calculation stage was reduced from 0.30 s to 0.03 s." Identify at least three problems using the source's pipeline example.

Answer and explanation

Big-O describes growth under a model, not a maximum duration in milliseconds. Constants, input sizes, implementation, hardware, and other stages are missing. The cited stage times are hypothetical, not measured evidence. Even accepting them as a model, the unchanged 0.60 s read/validation and 0.10 s report stages yield total time 0.73 s instead of 1.00 s: only about 1.37-fold overall speedup. Accuracy, data validity, and output preservation must still be established.

5.13. Readiness checklist

A student ready for the searching-and-sorting unit should be able to identify hidden scans, distinguish average-case from amortized reasoning, track all recursive work separately from peak live storage, retain independent problem dimensions, normalize benchmark batches correctly, and explain why speed is subordinate to the stated output and accuracy contract.

Use the source examples to practice those explanations aloud. The goal is not to guess a complexity label from punctuation; it is to show the computation that justifies the claim.

6 Synthesis and Glossary

Lecture-slide alignment: Week 11 source PDF, p. 84; p. 87.

6.1. A complete explanation of efficiency

Evidence to report

For an unchanged record of n finite readings and a valid width w, the direct method returns $n - w + 1$ full-window means and performs $w(n - w + 1)$ selected additions. The running method returns the same number of means and performs $2n - w$ selected additions/subtractions. Both preserve the input and require storage for the output. The source benchmark observes their times for one declared fixture with $w = n/4$. The cancellation test shows that algebraic equivalence does not by itself establish numerical equivalence on every input.

This paragraph is stronger than "the second program is faster" because it states the task, growing dimensions, counts, preservation requirements, observation scope, and limitation.

6.2. Quick-reference comparisons

Pattern under the source's model	What to remember
Repeated prefix concatenation	Quadratic cumulative copying; linear peak result-list storage.
Repeated front removal	Triangular tail shifts; initial copy remains part of the total work.
List membership for all installed records	Separate q and m; do not omit output work at $m = 0$.
Index constructed once	Include first-use construction and later maintenance; qualify hashing.
Tail-slice recursion	Copies may accumulate in both total work and peak live storage.
Balanced disjoint recursion	Total tree size differs from maximum depth.
Cached recursion	Fresh computation and warm retrieval are different workloads.
Moving-window means	Keep n and w; test floating-point error after algebraic reuse.
Timing a callable	Normalize batch time by calls, retain trials, and state what is excluded.

6.3. Terminology

Input size.
A quantity that grows with the task, such as record length n, window width w, installed-ID count q, approved-ID count m, or number of channels. Several independent sizes may be necessary.

Analysis contract.
The declaration of required output, valid domain, selected operations, input case, and cost assumptions used to interpret an efficiency claim.

Selected-operation count.
The number of explicitly chosen events, such as equality tests, additions, or copied reference slots. It is not a complete CPU-instruction count.

Order of growth.
How a work or storage measure changes as the defined input size grows. The representation and the scaling relation between dimensions matter.

Big-O.
An eventual upper-bound description. It is not automatically a tight bound, a measured duration, or a worst-case label without an identified input case.

Tight growth, Θ.
An asymptotic description with matching upper and lower growth bounds under the stated model.

Hidden work.
Scans, allocations, shifts, comparisons, or helper computations that are not obvious from the length of an expression.

Shallow copy.
A new outer collection populated with references to the original elements. It does not recursively duplicate the referred-to objects.

Cumulative copying.
Total reference-copy work summed over the full execution, including allocations that do not remain alive afterward.

Peak live storage.
The greatest amount of relevant data simultaneously retained during an execution. Reachable references and unfinished frames determine what can coexist.

Auxiliary space.
Working storage apart from the supplied input and, in this unit's stated convention, the required returned output. Other conventions exist, so state the inclusion rule.

Amortized cost.
A bound for a sequence of operations distributed across the sequence, including the occasionally expensive operations.

Average or expected cost.
A cost interpreted using specified input or hashing assumptions. It is distinct from an amortized sequence argument.

Lookup index.
A prepared representation, such as a dictionary of approved keys, that supports later queries. Building and keeping it current have costs.

Invariant computation.
A calculation whose relevant inputs and meaning stay unchanged through a loop, so it may sometimes be computed once before repetition.

Local recursive work.
Work performed by one invocation outside its child invocations, including construction of arguments for those children.

Total call count.
The number of all invocations during execution, including completed siblings and base cases.

Recursion depth.
A longest active call chain. This document counts frames for stack-memory statements and identifies when an edge count differs by one.

Disjoint subproblems.
Smaller tasks processing separate parts of the original problem, such as the two index intervals in balanced summation.

Overlapping subproblems.
Repeated occurrences of the same smaller problem, such as the repeated $F(2)$ calls in the naive $F(4)$ tree.

Memoization or cache.
A mapping from a subproblem key to a previously computed result. Reuse is valid only when the key still identifies the same problem and stored result is correct.

Fresh versus warm cache.
A fresh request includes the work of computing missing states. A warm hit retrieves a value already present. The two are not interchangeable benchmark tasks.

Full window.
A consecutive block with all w required readings present. An n-reading record has $n - w + 1$ full windows for $1 \leq w \leq n$.

Running subtotal.
A stored sum updated as a window changes, rather than recomputed from scratch. Finite-precision error can persist in that stored state.

Cancellation stress test.
A deliberately chosen numerical case that exposes loss of small contributions when large values enter or leave an arithmetic calculation.

Doubling ratio.
A ratio of observed times or declared counts at sizes $2n$ and n. Keep the kind of quantity and scaling policy explicit.

Empirical order.
A finite-size estimate such as $\log(t(2n)/t(n))/\log 2$. It is not a proof of asymptotic complexity.

Batch-normalized time.
The duration of repeated calls divided by the number of calls in the batch. It does not mean dividing by the number of trials.

End-to-end cost.
The total across preparation, validation, calculation, output, and other relevant stages of the actual task.

Regression check.
A retained test that detects whether a later change reintroduces a known defect or violates a previously checked promise.

A Source Basis and Slide-by-Slide Coverage

A.1. Source basis and technical references

Primary teaching source. Yanwei Wang, *Programming for Engineers: Week 11 — Understanding Program Efficiency II*, Fall 2026, 87 physical PDF pages. This is the Week 11 combined deck previously produced in this conversation, not the Week 10 file appearing among the recent attachments. Its earlier path is `ENG101_Week11_LaTeX_Source/week11_full.pdf`.

Recovered companion. The earlier `examples/week11_models.py` provides the 26 function/-class definitions used in the source snippets. The definitions included in this project preserve their executable behavior. The source module, the original timing files, and a SHA-256 manifest are retained for auditing. New driver scripts, observed outputs, activity solutions, worked instrumentation, and tests are reading-note additions and are identified accordingly.

Historical measurements. The original `timing_raw.csv`, `timing_summary.csv`, and `timing_environment.json` are copied without changes. Counts and ratios derived from these records are distinguished from new measurements. The source tool named `tools/generate_data.py` was not among the recovered resources; the new reproduction tool supplied here is labelled as such.

Background named by the source slides. Wang's supplied manuscript *Programming for Engineers with Python*, v20260815, Appendix G.11–G.16, G.21–G.25, G.28 and G.31–G.33; and J. V. Guttag, *Introduction to Computation and Programming Using Python*, second edition, Chapter 9 and Section 13.1 for the caching connection. These are further reading named in the deck, not substituted sources for its original timing experiment.

A.2. What was expanded or clarified

Complete prose explanations, derivations of the printed counts, worked activity answers, cumulative practice, and the example drivers were written for this reading package. Schematics were redrawn in editable TikZ using the source relationships. The timing plot reads the original summary CSV, with no fit or invented measurement.

Slice-cost wording is qualified by the number of elements actually copied; selected counts are distinguished from whole-runtime work; amortized and expected costs remain separate; recursion depth is distinguished from total calls and retained list storage; and the width helper's checks are not represented as a complete numerical-data validation. These are explicit explanatory qualifications, not silent changes to the source implementations.

The source's arithmetic-budget and pipeline-duration examples remain hypothetical. Its numerical cancellation case remains a counterexample, without an unsupported claim of a stable production replacement. Optional permutations and preprocessing material remain optional. No deadline, weight, examination claim, or new assessment rule is added.

A.3. Map every physical source page to a reading section

Use the physical PDF page shown below, not the slide footer number. One reading section can expand several short slides, while some source concepts also appear in the glossary or worked practice.

Source page	Reading section	Notes page
1	About these notes	2
2	About these notes	2
3	About these notes	2
4	Two loops do not settle the answer	6
5	Two loops do not settle the answer	6
6	Two loops do not settle the answer	6
7	Write the analysis contract first	7
8	Familiar operations and their hidden work	7
9	Repeated concatenation copies growing prefixes	8
10	Repeated concatenation copies growing prefixes	8
11	Append reduces repeated work without changing the task	9
12	Amortized growth: spread the occasional resize over the sequence	10
13	Amortized growth: spread the occasional resize over the sequence	10
14	Removing from the front repeats shifts	11
15	One loop can contain a complete membership scan	13
16	One loop can contain a complete membership scan	13
17	Construct and reuse a dictionary membership index	14
18	Construct and reuse a dictionary membership index	14
19	Construct and reuse a dictionary membership index	14
20	Test what must remain unchanged	15
21	Look inside methods and move only invariant work	15
22	Look inside methods and move only invariant work	15
23	Look inside methods and move only invariant work	15
24	Sum dependent workloads instead of multiplying loop bounds	17
25	Sum dependent workloads instead of multiplying loop bounds	17
26	Output requirements, recap, and exit-ticket answers	19
27	Output requirements, recap, and exit-ticket answers	19
28	Output requirements, recap, and exit-ticket answers	19
29	Recursion is a program structure, not a complexity class	20
30	Recursion is a program structure, not a complexity class	20
31	Recursion is a program structure, not a complexity class	20
32	Total nodes and maximum active frames are different counts	20
33	Index-recursive summation: keep the same list	21
34	Tail-slice summation: count what happens before the call	22
35	Tail-slice summation: count what happens before the call	22
36	Peak live memory: the waiting frames retain their tails	22
37	Compare the sum implementations under one contract	23
38	One halving call with constant local work	24
39	A logarithmic number of calls can still copy linear data	25
40	Follow one half, or process both disjoint halves	26
41	Follow one half, or process both disjoint halves	26

Source page	Reading section	Notes page
42	Full local scans at every tree level	27
43	Full local scans at every tree level	27
44	Fibonacci: the subproblems overlap	28
45	Fibonacci: the subproblems overlap	28
46	Fibonacci: the subproblems overlap	28
47	Memoization: make reused subproblems explicit	30
48	Memoization: make reused subproblems explicit	30
49	Memoization: make reused subproblems explicit	30
50	The cache key must identify the same problem	31
51	Worked activity, practical limits, and exit-ticket answers	32
52	Worked activity, practical limits, and exit-ticket answers	32
53	Worked activity, practical limits, and exit-ticket answers	32
54	Predict the scaling before measuring it	33
55	Predict the scaling before measuring it	33
56	Predict the scaling before measuring it	33
57	Predict the scaling before measuring it	33
58	Specify every full-window mean	34
59	Direct calculation: recompute every window	34
60	Direct calculation: recompute every window	34
61	Reuse the overlap: derive the running update	36
62	Reuse the overlap: derive the running update	36
63	Reuse the overlap: derive the running update	36
64	Test values, output length, boundaries, and non-mutation	37
65	A faster algebraic update can lose required information	38
66	Work, workspace, and returned output	39
67	Plan the timing experiment around a fixed question	39
68	Plan the timing experiment around a fixed question	39
69	Read the original measurements as a recorded experiment	40
70	Read the original measurements as a recorded experiment	40
71	What the timing supports, and what it does not	42
72	Operation budgets and multiple channels	42
73	Operation budgets and multiple channels	42
74	End-to-end speedup includes unchanged stages	43
75	Review an improvement, prepare the lab, and report evidence	43
76	Review an improvement, prepare the lab, and report evidence	43
77	Review an improvement, prepare the lab, and report evidence	43
78	Exit-ticket answers and the next unit	44
79	Familiar operations and their hidden work	7
80	A bound for the toy resizing strategy	45
81	Factorial growth: enumerating all orderings	45
82	Preparation before repeated searches	46
83	Reproducibility without silently changing the experiment	47
84	Synthesis and glossary	53
85	Source basis and technical references	56
86	Source basis and technical references	56
87	Exit-ticket answers and the next unit	44

Bibliography

[1] Y. Wang. *Programming for Engineers: Week 11 — Understanding Program Efficiency II*. ENG 101, School of Engineering, Nazarbayev University, Fall 2026. Primary 87-page slide deck and recovered companion files.

[2] Python Software Foundation. *timeit — Measure execution time of small code snippets*. Technical reference for callable timing, batch normalization, repeated trials, and garbage-collection behavior. https://docs.python.org/3/library/timeit.html. Consulted 15 September 2026.

[3] Python Software Foundation. *Design and History FAQ*. Technical reference for list representation and dictionary behavior. https://docs.python.org/3/faq/design.html. Consulted 15 September 2026.

[4] Python Software Foundation. *sys — System-specific parameters and functions*, entry for `getrecursionlimit()`. https://docs.python.org/3/library/sys.html. Consulted 15 September 2026.

[5] Python Software Foundation. *Floating-Point Arithmetic: Issues and Limitations*. Technical reference for interpretation of the source's cancellation example. https://docs.python.org/3/tutorial/floatingpoint.html. Consulted 15 September 2026.